



An efficient framework for structured material particle generation in multi-context modeling

Zenan Huo¹ · Xiangcou Zheng² · Michel Jaboyedoff¹ · Yury Podladchikov¹ · Gang Mei³ · Xiong Tang^{4,5}

Received: 27 August 2025 / Accepted: 30 September 2025 / Published online: 17 October 2025
© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2025

Abstract

High-fidelity Material Point Method (MPM) simulations are particularly sensitive to preprocessing quality. This paper presents a novel Julia-based framework for rapid generation of high-quality structured material particles for 2D/3D and complex-terrain models. The framework performs partition recognition to assign heterogeneous properties and discretizes complex terrain based on known subsurface structures or failure surfaces predicted by the sloping local base level (SLBL). To inform the selection of an appropriate initial grid spacing, theoretical upper bounds are established for the geometric discretization error and for the property error associated with the three-partition model. We validate the framework on benchmark cases and show that fine geometric features are accurately preserved. Excluding data loading and writing, up to 100 million structured material particles are generated within 10 s. The framework supports algorithm composition, requires no volumetric meshing, and enables rapid resolution sweeps during preprocessing. MPM simulations, including a landslide case and a reproducible parametric study integrated with SLBL, further demonstrate the framework's robustness and broad applicability. The framework thus provides an efficient, general-purpose preprocessing solution for MPM, with potential for transfer to other particle-based methods.

Keywords MPM · Structured material particle generation · Multi-context modeling · Julia language

1 Introduction

The Material Point Method (MPM) is a numerical simulation approach introduced by Sulsky et al. [26]. It integrates the advantages of both Lagrangian and Eulerian perspectives, making it particularly effective in handling large

deformations, fractures, and complex material behaviors. In the field of geotechnical engineering, MPM has demonstrated significant advantages in simulating highly non-linear problems such as landslides, where it can accurately capture large-scale terrain changes and material flow [5, 7, 14, 41, 42]. In contrast, mesh-based methods such as the

Xiangcou Zheng, Michel Jaboyedoff, and Yury Podladchikov contributed equally to this work.

✉ Xiangcou Zheng
xczheng@szu.edu.cn

Zenan Huo
zenan.huo@unil.ch

Michel Jaboyedoff
Michel.Jaboyedoff@unil.ch

Yury Podladchikov
Yury.Podladchikov@unil.ch

Gang Mei
gang.mei@cugb.edu.cn

Xiong Tang
tangxiong19@mails.ucas.ac.cn

¹ Institute of Earth Sciences, University of Lausanne,
1015 Lausanne, Switzerland

² College of Civil and Transportation Engineering, Shenzhen
University, Shenzhen 516080, China

³ School of Engineering and Technology, China University of
Geosciences (Beijing), Beijing 100083, China

⁴ Institute of Mountain Hazards and Environment, Chinese
Academy of Sciences, Chengdu 610041, China

⁵ University of Chinese Academy of Sciences, Chinese
Academy of Sciences, Beijing 100049, China

Finite Element Method (FEM) suffer from mesh distortion and failure in such scenarios. In computer graphics, MPM has also been widely adopted for simulating complex physical phenomena and for modeling flexible multi-material and multi-physics interactions [19, 33].

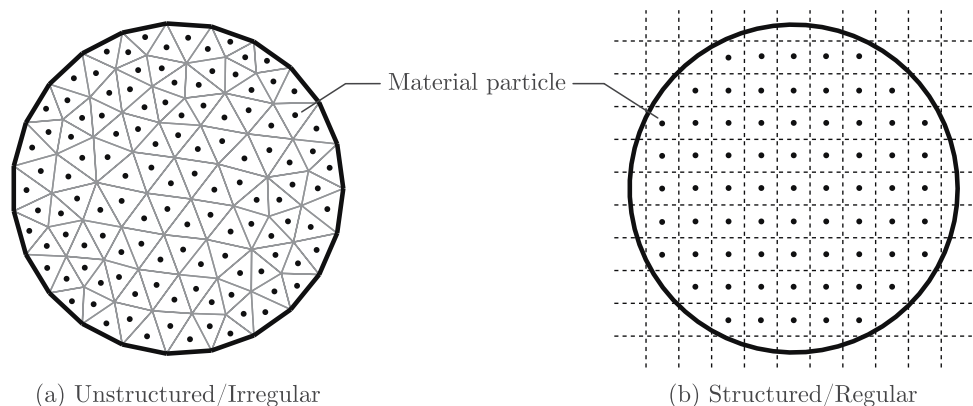
High-fidelity computational models are essential for capturing complex physical phenomena but often entail significant geometric complexity. This places high demands on preprocessing tools, particularly in multi-context modeling, where a unified framework must accommodate varying geometries, boundary conditions, and resolutions. Although MPM offers advantages in handling large deformations, its application remains limited. Researchers often rely on simplified models for validation, and the relatively low efficiency of explicit MPM constrains its use in large-scale simulations. More critically, the lack of general and efficient particle generation tools presents a bottleneck in high-resolution modeling workflows. Similar challenges were faced in the early development of FEM, which were addressed through the development of robust meshing tools such as *Gmsh* [12], *TetGen* [28], and other commercial software. These tools support structured and unstructured mesh generation, boundary adaptability, and local refinement, enabling broad applicability of FEM in complex engineering problems. In contrast, MPM still lacks comparable preprocessing capabilities, limiting its scalability and flexibility in practical applications.

MPM discretization results are generally categorized as unstructured (irregular) or structured (regular), as illustrated in Fig. 1. A straightforward approach to structured particle generation involves defining an axis-aligned bounding box (AABB) around the computational domain, then generating uniformly distributed particles within this box and filtering out those outside the model. However, this method has significant limitations: (1) in 2D, models with internal voids require additional algorithms for particle inclusion checks, (2) in 3D, ray-casting method is commonly used to test whether a given particle is inside an entity, the naive implementation often suffers from poor performance [30].

A more general approach involves using external preprocessing software to mesh the computational domain before generating particles. In 3D, the model is typically tetrahedralized, and structured or unstructured particles are then distributed within each tetrahedron. Software tools such as *CAD2Point* [39] and *PyCBG* [29] follow this procedure. It benefits from a well-defined workflow and existing meshing tools but comes with drawbacks: tetrahedral meshing incurs high computational costs, and the resulting particles are often unstructured, requiring additional steps to ensure a structured distribution. Another key application of MPM is the simulation of complex terrain, where the computational model is often derived from Digital Elevation Model (DEM) data. In such cases, direct particle filling within the model is possible without requiring prior meshing. Fernández et al. [11] proposed a DEM-based discretization scheme that incorporates multiple geological material classifications and was successfully used to simulate the Daguangbao landslide in China. You et al. [38] further refined this approach by considering geological features and structures. Meanwhile, Zhang et al. [40] introduced a novel method for rapidly generating particles from Standard Triangle Language (STL) files, significantly improving generation efficiency. Relevant techniques have also been explored in computer graphics, although not directly aimed at structured MPM particle generation. Techniques such as GPU-based voxelization [31] and Bounding Volume Hierarchies (BVH) [21, 35], originally developed for real-time rendering and collision detection, can be repurposed to accelerate point-in-solid queries.

From the aforementioned review, it is evident that no mature and efficient structured particle generation scheme currently exists to handle a wide range of application scenarios. To fill this gap, we propose a unified framework that supports complex 2D and 3D geometries, including terrain models derived from DEM data and other specialized inputs. Another key contribution is its exceptional computational efficiency. Since particle discretization does not require real-time performance and should not become a

Fig. 1 Discrete results of unstructured and structured particles



bottleneck, our methods are designed to avoid GPU dependence while still enabling fast particle generation even on consumer-grade or portable hardware. When high-resolution discretization completes in seconds, the generation cost becomes negligible relative to the overall MPM simulation time, resulting in a seamless workflow. The implementation, *MaterialPointGenerator.jl*, is written in Julia, leveraging the language’s performance and usability to support rapid structured particle generation for both research and practical applications.

The structure of this paper is organized as follows: Section 2 provides a brief overview of Julia’s key features, followed by an introduction to MPM principle and computational procedures, as well as an overview of algorithms involved in particle generation. Section 3 details the discretization algorithms and implementation strategies of the framework. Section 4 presents case studies that demonstrate the applications of the framework and discusses discretization accuracy, efficiency, and applicability to different scenarios, while Section 5 concludes the paper.

2 Background

The proposed framework is implemented in Julia, with its key features first introduced in comparison to other programming languages. We then summarize the governing

equations and computational steps of MPM, where p denotes the discretized material particles. Lastly, we review algorithms for determining whether a point lies inside a solid, along with their evolution and relevance to the algorithms in our framework.

2.1 The Julia language

Julia is a high-level, high-performance programming language specifically designed for scientific computing [2]. Unlike other dynamic languages such as Python and MATLAB, Julia achieves computational performance comparable to C/C++ and Fortran, while maintaining an intuitive and user-friendly syntax. Figure 2 selects six benchmark tasks: `iteration_pi_sum` evaluates floating-point operations and loop efficiency; `matrix_statistics` tests array processing; `print_to_file` measures I/O performance; `recursion_fibonacci` and `recursion_quicksort` assess function call overhead and recursion efficiency; `userfunc_mandelbrot` examines complex numerical computation and control flow handling. This efficiency is primarily attributed to its Just-In-Time (JIT) compilation based on LLVM, which enables the generation of highly optimized machine code. Furthermore, Julia employs multiple dispatch, a powerful paradigm that allows function specialization based on argument types, thereby ensuring both code readability and execution efficiency.

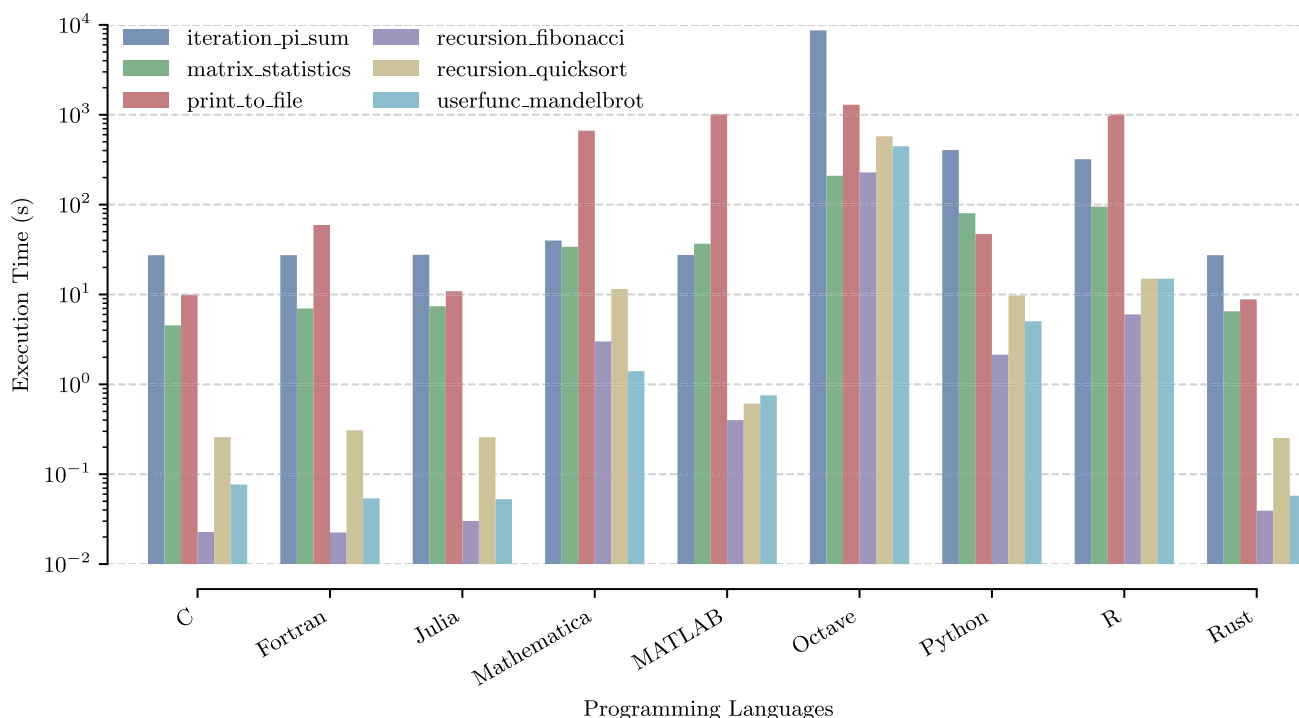


Fig. 2 Julia micro-benchmarks results. The benchmark is designed to evaluate the performance of identical algorithms and coding patterns implemented in different programming languages, excluding compilation time. More information can be found on the official Julia website [4]

Julia provides a modern package ecosystem and an efficient dependency management system, ensuring reproducibility and portability in scientific research. The balance between performance and usability has contributed to the increasing adoption of Julia in both academia and industry, particularly in fields that require high-performance numerical computing.

In this work, we have chosen Julia to develop an efficient structured particle generation framework. Julia is particularly well-suited for implementing algorithms without legacy constraints, while still ensuring excellent computational efficiency. One of its advantages is the ability to seamlessly integrate existing libraries from other programming languages, avoiding redundant development efforts. For instance, our framework incorporates *trimesh*, a widely used Python-based mesh processing library [8], through *PythonCall.jl* [25]. This integration allows users to interact with Python libraries directly within the Julia environment, without requiring manual dependency management or complex installations—everything is handled automatically with a single command. Furthermore, our framework is designed for cross-platform compatibility, supporting both x86 (Windows/Linux) and ARM (macOS) architectures with a straightforward installation process. To enhance usability and flexibility, we leverage multiple dispatch, a core feature of Julia. For example, the function `dem2particle` supports multiple input arguments—differing in argument types or quantity—allowing users to achieve different functionalities while maintaining a clean and intuitive API. These design choices ensure that our framework is not only efficient and scalable, but also accessible to a broad user base across different computing environments.

2.2 Overview of the MPM

MPM combines Lagrangian and Eulerian perspectives by representing the model using material particles that carry physical quantities such as mass, velocity, and stress, and move with the deforming body. At the same time, a fixed background grid is introduced to solve the governing equations, effectively avoiding mesh distortion issues. The momentum conservation equation in single-phase MPM is given by

$$\rho a = \nabla \cdot \sigma + \rho b, \quad (1)$$

where ρ denotes the material density, acceleration a is the primary unknown variable, $\nabla \cdot \sigma$ represents the divergence of the stress tensor corresponding to internal forces, and b is the body force. Large strains are presented through the deformation gradient

$$F = \frac{\partial x}{\partial X}, \quad L = \nabla v, \quad D = \frac{1}{2}(L + L^T), \quad (2)$$

where L is the velocity gradient and D the rate-of-deformation tensor. For elastoplastic materials a multiplicative split, $F = F^e F^p$ separates recoverable and permanent deformation. The elastic response is written in rate form using an objective (e.g. Jaumann) stress rate:

$$\overset{\nabla}{\sigma} = \mathbb{C} : D^e, \quad D^e = D - D^p, \quad (3)$$

where $\mathbb{C}$ is the fourth-order elasticity tensor and D^p is governed by an appropriate yield function and flow rule. The coupling of Eqs. (1)–(3) yields a fully nonlinear, closed-form representation of a continuum's dynamic behavior, while the separation between material particles and the background grid mitigates mesh distortion.

The weak form of the momentum conservation equation is expressed as:

$$\int_{\Omega} w \cdot \rho a \, d\Omega = - \int_{\Omega} \nabla(w) : \sigma \, d\Omega + \int_{\Omega} w \cdot \rho g \, d\Omega + \int_{\partial\Omega_{\tau}} w \cdot \tau \, dS, \quad (4)$$

where w denotes the test function defined in the appropriate function space, and τ corresponds to the external traction forces prescribed on the boundary $\partial\Omega_{\tau}$.

The overall solution process is illustrated in Fig. 3. First, physical quantities such as mass and momentum are transferred from particles to the background grid (P2G, Fig. 3a), which is governed by

$$m_i^t = \sum_p N_i(X_p^t) m_p, \quad (5)$$

$$(mv)_i^t = \sum_p N_i(X_p^t) (mv)_p^t, \quad (6)$$

$$F_i^{\text{ext},t} = \sum_p N_i(X_p^t) m_p b(X_p^t), \quad (7)$$

$$F_i^{\text{int},t} = - \sum_p V_p^t \sigma_p^t \partial N_i(X_p^t), \quad (8)$$

$$F_i^{\text{tot},t} = F_i^{\text{ext},t} + F_i^{\text{int},t}. \quad (9)$$

Here, X_p^t denotes the spatial position of particle p at time t . The particle properties including mass, velocity, and volume are denoted by m_p , v_p , and V_p , respectively. The shape function N_i and its spatial derivative ∂N_i define the interpolation and its gradient at the grid node i . The force terms $F_i^{\text{ext},t}$, $F_i^{\text{int},t}$, and $F_i^{\text{tot},t}$ correspond to the external, internal,

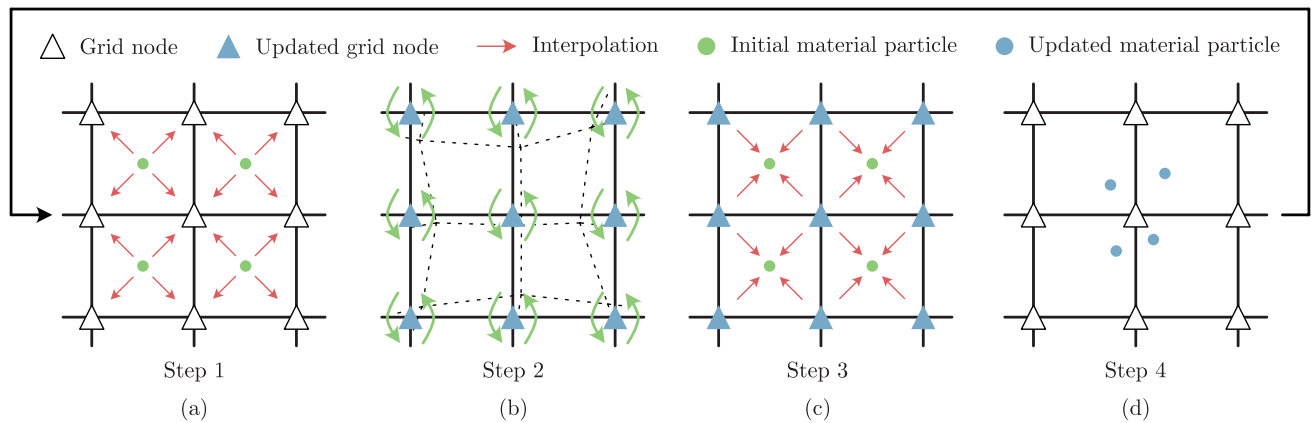


Fig. 3 Typical 2D MPM numerical cycle. **a** Particle-to-grid (P2G) mapping transfers particle information to the grid. **b** Physical quantities are computed on the grid. **c** Grid-to-particle (G2P) mapping inter-

polates results back to particles. **d** The grid is reset for the next iteration, while particle states are preserved

and total nodal forces, respectively, where b denotes the body force field applied to the particles.

Next, the momentum equation is solved on the grid to compute nodal accelerations and velocities (Fig. 3b), which can be expressed as:

$$(m\tilde{v})_i^{t+\Delta t} = (mv)_i^t + \Delta t F_i^{\text{tot},t}, \quad (10)$$

$$\tilde{v}_i^{t+\Delta t} = (m\tilde{v})_i^{t+\Delta t} / m_i^t, \quad (11)$$

where Δt represents the time integration step. The updated information is then interpolated back to the particles to update particle positions $X_p^{t+\Delta t}$, velocities $v_p^{t+\Delta t}$, incremental deformation gradient $\Delta F_p^{t+\Delta t}$, and deformation gradient $F_p^{t+\Delta t}$ (G2P, Fig. 3c):

$$X_p^{t+\Delta t} = X_p^t + \Delta t \sum_i N_i(X_p^t) \tilde{v}_i^{t+\Delta t}, \quad (12)$$

$$v_p^{t+\Delta t} = \sum_i N_i(X_p^t) \tilde{v}_i^{t+\Delta t}, \quad (13)$$

$$\Delta F_p^{t+\Delta t} = \Delta t \sum_i \partial N_i(X_p^t) \tilde{v}_i^{t+\Delta t}, \quad (14)$$

$$F_p^{t+\Delta t} = (I + \Delta F_p^{t+\Delta t}) F_p^t. \quad (15)$$

At the end of each time step, particle stress is updated according to the selected constitutive model. The background grid is reinitialized and discarded, while the updated particle state is preserved for the subsequent iteration (Fig. 3d). By combining particle-based state tracking with

grid-based computations, MPM achieves both robustness and flexibility in handling large deformations and complex contact interactions. More details can be found in Zheng et al. [43] and Huo et al. [14], they are omitted for the purpose of simplification.

2.3 Point-in-solid algorithms

For unstructured grids, particles are typically generated by filling pre-meshed elements. In contrast, structured grid-based particle generation requires only a closed surface mesh, eliminating the need for volumetric meshing. As a result, the discretization problem is simplified to identifying which points lie inside the model. We typically first construct the corresponding axis-aligned bounding box based on the computational model and its resolution (particle spacing), followed by uniformly filling it with particles. At this stage, an efficient point-in-solid algorithm is required to identify particles located inside the model. Assuming the computational model is a watertight solid composed of triangular faces, a widely used approach is based on the Jordan Curve Theorem: a ray is cast from the test particle in an arbitrary direction, and the number of intersections with the triangular faces of the model is recorded. If the intersection count is odd, the point is considered inside; otherwise, it is outside (i.e., the ray-casting method) [27]. From an implementation perspective, the simplest approach is to traverse all triangles, resulting in a time complexity of $\mathcal{O}(m)$, where m is the number of triangular faces. However, as model complexity increases and high-resolution MPM simulations require particle counts in the millions, the computational cost of this naive implementation becomes prohibitive.

Optimized strategies based on the Jordan Curve Theorem, as well as alternative approaches, mainly include: (i)

grid-based methods such as voxelization [6]; (ii) acceleration techniques leveraging spatial data structures like Octrees and Bounding Volume Hierarchies (BVH) [35, 36]; and (iii) GPU-accelerated methods [20]. Since real-time performance is not required for particle generation in numerical simulations, our goal is to design a simple yet efficient point-in-solid algorithm suitable for general-purpose computing platforms, deliberately avoiding GPU-based solutions. While GPU acceleration can speed up ray-casting when used with spatial data structures, alternative optimization strategies also exist. For example, in the work of Zhang et al. [40], the triangular faces of the model are projected onto a two-dimensional plane for point-in-solid testing. This dimensionality reduction makes intersection computations significantly more efficient in 2D case than in 3D case, thereby reducing overall computational costs. Moreover, this method avoids reliance on complex data structures or preprocessing, making it well-suited to structured particle generation workflows and easily parallelizable.

3 Methodology

This section proposes a unified framework for structured material particle generation across 2D, 3D, and complex terrain models. The proposed framework does not impose stringent restrictions on input file formats, as various model formats, such as .STL, .OBJ, .PLY, .XYZ, .OFF, .VTP, and .CSV, can be converted within the framework or via the extensive ecosystem of third-party tools. For concreteness, we use the STL (Stereolithography) format as a representative example, which strikes a balance among broad applicability, binary compactness, and a well-defined public specification. For 2D models, the framework adopts a conventional rasterization approach; note that rasterization is essentially a special case of 3D voxelization, where all vertex z -coordinates are set to zero. Consequently, both processes are hereafter referred to uniformly as “voxelization.” Complex 3D terrains are handled using a distinct algorithm, as detailed in Section 3.2. For landslide post-failure propagation scenarios, the sloping local base level (SLBL) method is incorporated to predict potential failure surfaces and directly construct material particle representations of the sliding body. Given that MPM is frequently employed in geotechnical engineering to simulate materials with spatially varying properties, the generated material particles are further categorized to facilitate the assignment of different physical parameters, boundary conditions, or constitutive models, as discussed in Section 3.3. The framework integrates the algorithms and workflows described in Sections 3.1–3.3, covering the majority of structured material

particle generation needs. For highly complex geometries, conversion to a STL file followed by the procedures in Sections 3.1.1–3.1.3 is recommended. Finally, Section 3.4 introduces quantitative evaluation metrics to assess the discretization quality and efficiency.

3.1 Generic two- and three-dimensional models

Structured material particle generation for 2D models generally does not pose significant performance challenges. In the framework, different strategies are adopted based on model complexity: (i) for regular and simple 2D geometries, efficient algorithms such as analytical functions or winding number methods are used; (ii) for models provided in STL format, a voxelization-based approach is applied. In contrast, particle generation for complex 3D models requires careful algorithmic design, as high-resolution particle distributions can easily lead to excessive preprocessing times. To balance usability and performance, our framework employs a 3D voxelization scheme and further introduces an optimized projection-based algorithm, which enables fast computation and memory friendliness.

3.1.1 Winding number

The winding number algorithm is a classical method in computational geometry for determining whether a point lies inside a polygon. The fundamental principle is to compute the net number of times the polygon boundary winds around the query point, referred to as the winding number. This number depends on the ordering of polygon vertices, typically assumed to be counterclockwise. A nonzero winding number indicates that the point lies inside the polygon, while a value of zero indicates that it lies outside. In practical implementation, the algorithm examines whether each edge of the polygon crosses the horizontal line defined by the query point's y -coordinate. For each crossing, the algorithm considers both the direction of traversal (upward or downward) and the relative position of the point with respect to the edge (left or right), which is typically determined using the cross product. Based on these conditions, the winding number is incremented or decremented accordingly. Ultimately, the final winding number determines the point's location relative to the polygon.

Let a polygon P be defined by an ordered sequence of vertices $\{p_0, p_1, \dots, p_{n-1}\}$, where $p_i = (x_i, y_i)$, and $p_n = p_0$ to form a closed boundary. Let $q = (x_q, y_q)$ be the query point. The winding number $w(P, q)$ represents the net number of times the polygon boundary P winds around the point q . Mathematically, it is defined as:

$$w(P, q) = \frac{1}{2\pi} \oint_P d\theta, \quad (16)$$

where θ denotes the polar angle of each boundary point relative to q , and the integral $\oint_P d\theta$ represents the total angular change along the closed path defined by P .

The winding number algorithm handles self-intersecting and holed polygons well, making it suitable for complex geometries. However, floating-point errors near edges or vertices can cause misclassifications, requiring additional robustness. With $\mathcal{O}(n)$ time complexity, its efficiency declines for polygons with many edges. Thus, in our framework, it is used only for simple inclusion tests, where it remains reliable.

3.1.2 Voxelization

A voxel is the volumetric counterpart of a pixel in 2D space, typically represented as a cube. It stores attribute information at a specific location in 3D space, such as color, density, or material type, and provides an efficient way to represent and process volumetric data. Voxels are widely used in computer graphics (e.g., volume rendering and terrain generation), medical imaging (e.g., 3D reconstruction from CT or MRI scans), 3D modeling, robotic perception, scientific computing, and virtual reality [9, 10]. The requirements of our framework align well with the intuitive nature and structural adaptability of voxels. Each cubic voxel can naturally correspond to a background grid cell in MPM. This allows us to reformulate the particle generation task as follows: (i) voxelize the computational domain based on the resolution of the background grid; (ii) uniformly fill each voxel with material particles according to the pre-defined number of particles along each spatial direction. Moreover, the attributes of the material particles can be directly assigned from the voxel properties. This voxel-based approach offers a significant advantage: once a voxelized model with assigned attributes is obtained, both structured particle filling and

property assignment become computationally trivial. Given that voxelization algorithms are mature and capable of handling highly complex geometries, this strategy provides a general solution for preparing MPM models across a wide range of scenarios.

Three-dimensional models are often represented by triangular faces, requiring surface voxelization followed by internal voxel filling. We generate a uniform voxel grid within a cubic region containing all triangular faces, with nodes as voxel centers. Surface voxelization involves detecting voxel-face intersections, but testing each face can cause performance bottlenecks, especially with many faces and high resolution. Thus, we use a sampling-based surface voxelization approach, which is fast and robust on CPU, with accuracy depending on sampling density.

Let the triangular surface mesh be $\mathcal{M} = \{\tau_i\}_{i=1}^n$ with vertices $v_{i,0}, v_{i,1}, v_{i,2} \in \mathbb{R}^3$. Choose an isotropic voxel size $h > 0$ and a global grid origin x_0 . For each triangle $\tau \in \mathcal{M}$ we use barycentric coordinates:

$$x(u, v) = (1 - u - v)v_0 + uv_1 + vv_2, \quad (17)$$

where $u, v \geq 0, u + v \leq 1$. Select a step length $\Delta = h$ and generate the discrete barycentric set

$$\mathcal{S}_\tau = \{(u, v) \mid u = p\Delta^*, v = q\Delta^*, p, q \in \mathbb{N}_0, u + v \leq 1\}, \quad (18)$$

$$\Delta^* = \frac{\Delta}{\|(v_1 - v_0) \times (v_2 - v_0)\|^{1/2}}. \quad (19)$$

Each barycentric coordinates $(u, v) \in \mathcal{S}_\tau$ yields a sample point x_s , as illustrated in Fig. 4. A sample point is mapped to the voxel grid by

$$k = \left\lceil \frac{x_s - x_0}{h} \right\rceil \in \mathbb{Z}^3, \quad (20)$$

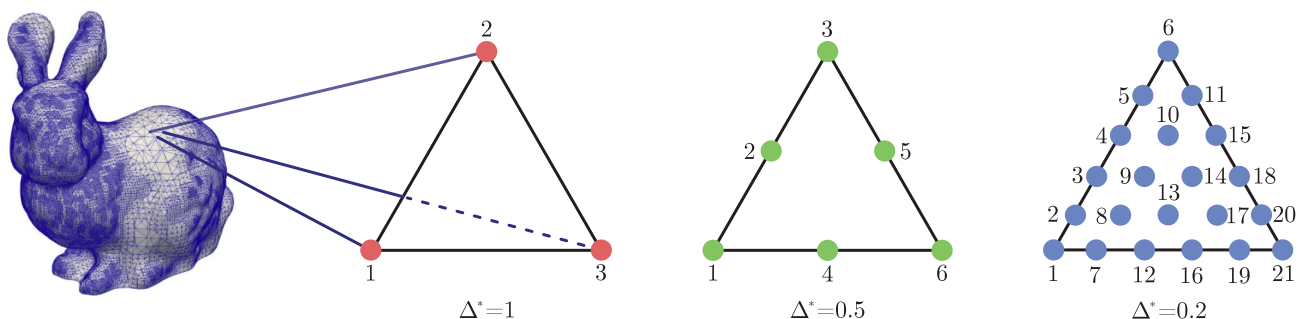


Fig. 4 Sampling-based surface voxelization pipeline. The input is a model represented in STL format. Taking one of its triangular faces as an example, the sampling density of points (represented as red, green,

and blue circles) is controlled by selecting different Δ^* -values. These sampling points are subsequently aligned to the voxel grid

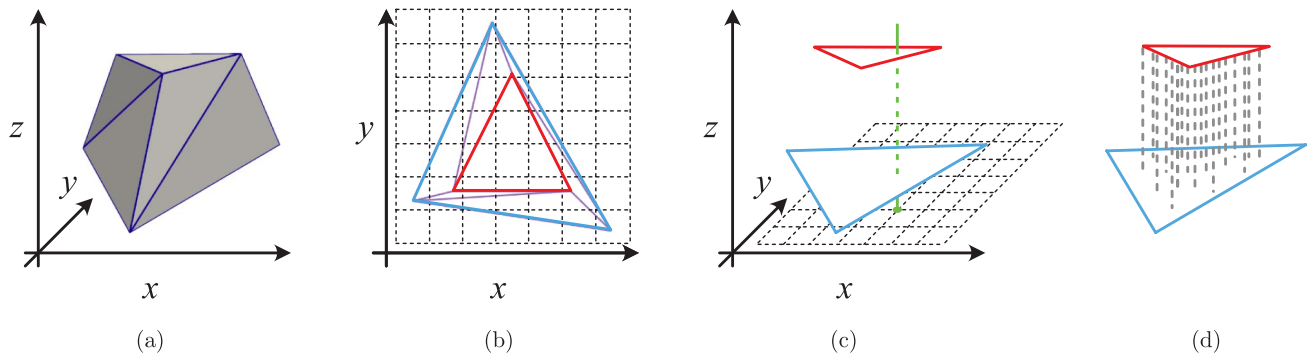


Fig. 5 Schematic illustration of optimized projection-based particle generation for geometries with or without cavities. **a** The input model to be discretized. **b** All triangular faces are projected onto the xy -plane, and a structured grid is generated based on the bounding box of the projection. Taking the top and bottom surfaces as an example (high-

lighted in red and blue), a grid node n located within both projected triangles is identified. **c** A ray is cast from node n along the $+z$ direction, and its intersections with the mesh surfaces are computed (indicated by green lines). **d** Particles are then uniformly placed along the segment between the two intersection points to fill the interior

Insert k into a hash set $\mathcal{V}$; duplicates are discarded in $\mathcal{O}(1)$ time. After processing all triangles, the voxel set

$$\mathcal{V} = \{k \in \mathbb{Z}^3 \mid k \text{ generated by some } \tau_i\} \quad (21)$$

forms a one-voxel-thick occupancy shell that faithfully approximates the model surface while completely avoiding triangle-voxel intersection tests. This forward sampling scheme is inherently parallelable, numerically robust, and its complexity is $\mathcal{O}(\sum_{\tau} |\mathcal{S}_{\tau}|)$, governed solely by the chosen sampling density rather than by spatial overlap queries, making it particularly attractive for rapid particle seeding.

After surface voxelization, a straightforward external marking strategy can be employed to accurately identify the interior of a closed mesh for volumetric filling. The process begins with a boundary extension of the original voxel grid $\mathcal{V}$, where a single layer of empty voxels is appended to the periphery of the domain to ensure global connectivity of the exterior space. Subsequently, a 3D morphological reconstruction (flood-based dilation) is performed, using the extended boundary as the seed to iteratively label all voxels that are accessible from the outside. Once the dilation converges, the complement of the labeled external region yields the set of voxels that are topologically isolated from the exterior, effectively defining the model's boundary. This interior mask is then mapped back to the original voxel resolution to complete the volumetric filling [15]. This method eliminates the need for internal seed identification and relies solely on topological connectivity analysis, enabling a robust and efficient filling process with linear-time complexity. However, this approach has a notable limitation: it cannot preserve internal cavities within the model. Furthermore, it requires the input mesh

to be strictly watertight, as any topological inconsistency may lead to erroneous results.

3.1.3 Optimized projection-based algorithm

To populate 3D geometries that may contain internal cavities, we implement an optimized projection-based algorithm for particle generation. This algorithm is conceptually similar to the approach reported by Szucki and Suchy [32]; Zhang et al. [40], but streamlined for higher computational throughput. First, all triangles are orthogonally projected onto the plane $z = 0$ (the model is assumed to satisfy $\min(z) > 0$). A uniform Cartesian grid is then constructed over the xy -plane, using the AABB from the projection and a pre-specified spatial resolution h . For each projected triangle, we identify the grid nodes encompassed by its planar footprint and record the mapping “node $\rightarrow$ triangle” indices in list L_1 . The search is accelerated by culling nodes outside the triangle's planar AABB and by an oriented-area (cross-product) test, which is equivalent to the barycentric-coordinate criterion. A preprocessing step builds list L_2 . For every node n_i , associated with triangles $\triangle ABC, \triangle DEF, \dots$, a ray is cast from n_i in the $+z$ direction; the intersection z -coordinates are computed and sorted in ascending order, yielding the sequence $[z_1, z_2, \dots]$ in L_2 . Material particles are subsequently seeded only within the intervals $(z_1, z_2), (z_3, z_4), \dots$, i.e., between successive odd-even index pairs—thereby leaving interior voids unfilled. The complete procedure is summarized in Algorithm 1 and Fig. 5.

Data: Surface mesh $\mathcal{T}$ ($z > 0$);
Grid spacing h .
Result: Interior particle set $\mathcal{P}$;

1. Projection & grid nodes
Project $\mathcal{T}$ to $z = 0$, obtain $\Pi(\mathcal{T})$;
Tessellate its AABB with uniform nodes $\mathcal{N}$ (with space h).

2. Node–triangle map L_1
foreach $\tau \in \Pi(\mathcal{T})$ **do**
 foreach node n in $\text{AABB}_{2D}(\tau)$ **do**
 if $n \in \tau$ **then**
 append τ to $L_1[n]$;
 end
 end
end

3. Ray–facet heights L_2
foreach n with $L_1[n] \neq \emptyset$ **do**
 foreach $k \in L_1[n]$ **do**
 add $z_k = \text{ray}(n) \cap \tau_k$ to $L_2[n]$;
 end
 sort $L_2[n]$;
end

4. Odd–even fill
 $\mathcal{P} \leftarrow \emptyset$;
foreach n with $L_2[n] = [z_1, \dots, z_m]$ **do**
 for $p = 1$ **to** $\lfloor m/2 \rfloor$ **do**
 for $z = z_{2p-1}$ **to** z_{2p} **step** h **do**
 add (n_x, n_y, z) to $\mathcal{P}$;
 end
 end
end
return $\mathcal{P}$;

Algorithm 1 Optimized projection-based algorithm

Although the current algorithm is highly parallelizable, for instance, during the maintenance of lists L_1 and L_2 and the particle filling stage, we found that even a single-threaded implementation is sufficiently fast. To enhance the robustness of the code, we opt to retain a single-threaded design and carefully address type instabilities that could significantly impact performance in Julia. A detailed evaluation of computational performance can be found in Section 4.

3.2 Complex terrain

For complex terrain models, data are typically acquired through airborne LiDAR, satellite radar, or other remote sensing techniques [24]. A common characteristic of these datasets is that they contain 3D point coordinates representing the surface of the modeled terrain, typically stored in interoperable

and easily convertible formats. Therefore, in this paper, we refer to all such datasets uniformly as digital elevation model (DEM) files, as they share a common processing workflow. Each point in a DEM represents a surface sample with 3D coordinates, where the z -value denotes elevation. To generate structured material particles, a natural approach is to (i) rasterize the DEM points in the xy -plane to produce a structured grid in 2D, and (ii) perform uniform filling along the z -axis, the complete method schematic is shown in Fig. 6. Owing to the feature of DEM data, this method enables efficient generation of structured particles without the need to address the point-in-solid problem. However, in some cases, terrain may be represented using surface models such as STL files, where surfaces are described by triangular meshes. In such scenarios, the general methodology described in Section 3.1 should be followed.

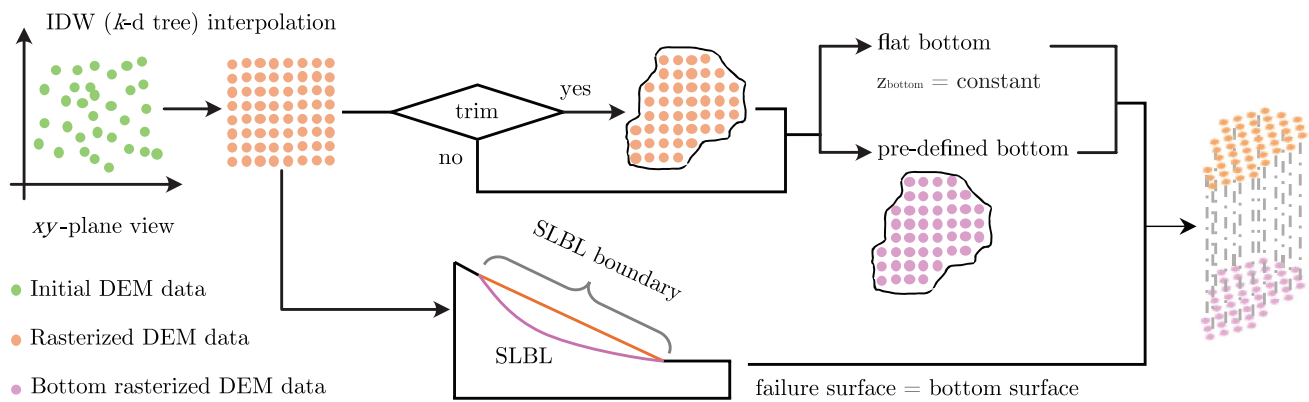


Fig. 6 Schematic illustration of structured particle generation for complex terrain (in the xy -plane view). Uniformly distributed particles are first obtained via interpolation and optionally trimmed to match the top surface shape (shown in orange). Three types of bottom surfaces are considered: (i) a constant-elevation flat plane, (ii) a non-planar surface

generated using the same interpolation process as the top surface, and (iii) a failure surface defined by the SLBL method. Particles are then uniformly filled between the top and bottom surfaces (gray dashed lines). The SLBL computation implicitly adjusts the surface shape, as indicated by the SLBL boundary

To perform rasterization on the 2D plane, we adopt inverse distance weighting (IDW) interpolation [3, 13]. A structured grid is first generated based on the spatial extent of the DEM and the pre-defined resolution, and each grid node is assigned an interpolated elevation value. IDW is simple, well-suited to DEM data, and can be easily replaced by more advanced methods such as spline or radial basis function (RBF) interpolation [1]. The accuracy of the resulting model depends largely on the quality of the original DEM data. For each grid node, a fixed number of nearby points are efficiently retrieved using a k -d tree, which greatly reduces the computational cost of neighbor searches. This rasterization method is robust and flexible, allowing us to control the resolution and domain boundaries. Unlike approaches that reconstruct a continuous surface from DEM data using triangulated meshes for interpolation, our method avoids additional algorithms and surface accuracy evaluation. Moreover, when the DEM resolution is high, it offers significantly better computational performance.

In practical applications of MPM simulations for complex terrains, two issues often arise: (i) the bottom surface of the model may not be flat, and (ii) the domain in the xy -plane may be an arbitrary polygon rather than a rectangle. Therefore, preprocessing is required when constructing models from DEM data. If the bottom surface is non-planar, it can be rasterized in the xy -plane using the same resolution as the top surface. Additionally, the domain boundary must be defined in the xy -plane, typically by specifying the polygonal region in counterclockwise order. Once both the top and bottom surfaces are represented on a common grid, material particles can be uniformly filled between them along the vertical direction at a pre-defined resolution. The rasterized points on the top and bottom surfaces have their x and y coordinates strictly aligned with the grid, with a spacing of h . The z -values of these points are obtained through

IDW interpolation. Prior to particle generation, each z -value is snapped to the nearest grid node to ensure structured particle placement.

For the landslide simulations using MPM, it is often necessary to define the sliding region based on a potential failure surface, which is then discretized into structured material particles. Although subsurface information is typically unavailable, several methods have been proposed to approximate failure surfaces. For example, the half-ellipsoid approximation has been extended to generate ellipsoidal and modified ellipsoidal failure surfaces. Another approach is the sloping local base level (SLBL) method, derived from the geomorphology concept of the base level, which supports automated processing and has been validated in several real-world cases [17, 18]. Our framework implements this method along with a set of advanced operations to facilitate the generation of sliding-region particles. Details of the SLBL method and its advanced usage are provided in Appendix A.

3.3 Particle property

The structured material particles generated in Sections 3.1–3.2 share uniform properties, as they are obtained solely through discretization based on the model's geometric information. However, in MPM simulations, a single scene may consist of multiple distinct objects or a single object subdivided into different partitions, each with its own set of numerical parameters. Similarly, complex terrain models may incorporate geological structures or be coupled with engineered structures. Therefore, it is necessary to address two general situations: models composed of multiple partitions and those involving terrain with internal heterogeneity.

For general 2D and 3D models, surface information is also represented using STL files. We utilize the open-source

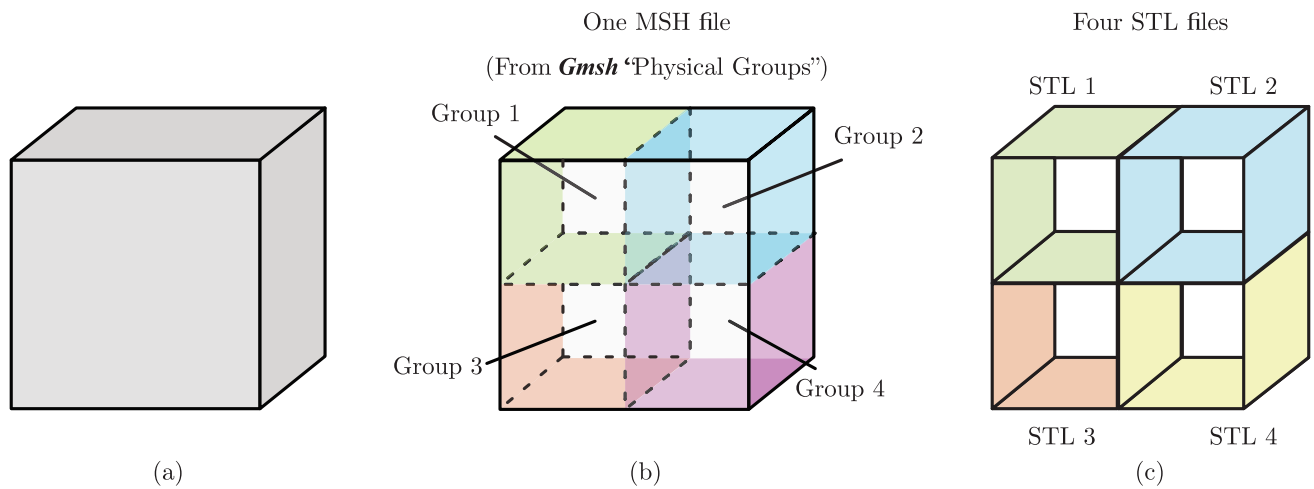


Fig. 7 Schematic of the property-based partitioning method: **a** example model to be partitioned; **b** partition labeling using the “Physical Groups” attribute in *Gmsh*; **c** alternative approach in which partitions

are exported as separate STL files. In case **c**, a common background grid is required for particle generation, and duplicate particles along partition boundaries should be removed after merging

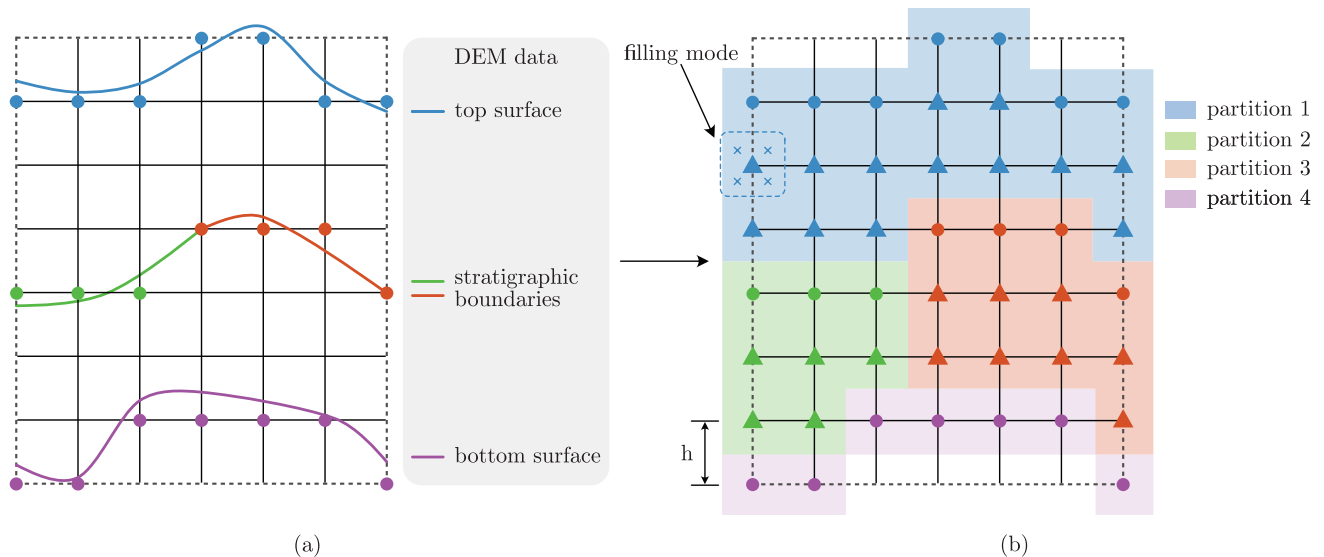


Fig. 8 Illustration of the discretization workflow for a terrain model including subsurface structure. Surface data are attached to grid nodes (circles), and structured particles are filled independently within each partition (triangles)

and efficient software *Gmsh* to import these STL files and incorporate partitioning information by constructing closed surfaces (in 2D) or enclosed volumes (in 3D). This step is analogous to the preprocessing workflows commonly employed in other numerical methods and results in an MSH file that contains both surface geometry and partitioning data. The inclusion of partitioning information is effectively equivalent to assigning labels to the boundary surfaces or volumes of individual subregions. Following the method described in Section 3.1, we first generate structured material particles based on the surface geometry. Then, for each material particle, we identify the partition boundary surface or volume it resides in and assign the corresponding partition label. Although our current implementation uses

the MSH file to encode partitioning information, the same procedure can be extended to other file formats capable of representing regional labels. In the MSH file, partitioning is defined in *Gmsh* through the use of “Physical Groups” (Fig. 7b). A more direct approach is to export each partition of the model as an individual STL surface file during the modeling stage (Fig. 7c). These files can then be discretized sequentially into structured material particles. In the final step, all particles are aligned and merged within a common background grid, with duplicate particles removed to ensure consistency across partitions.

In complex terrain models, material partitioning falls into two main categories: (i) subsurface geological zoning and (ii) structural zoning. Subsurface geological structures

are commonly derived from a combination of borehole sampling, geophysical surveys (e.g., seismic reflection, electrical resistivity), and geological mapping [44]. While borehole data offer high-resolution local insights, geophysical methods are more suitable for large-scale or deep subsurface interpretation. By integrating these heterogeneous datasets within advanced 3D geological modeling software, one can construct detailed geological models that delineate interfaces such as stratigraphic boundaries, faults, and the bedrock-overburden contact. These interfaces form the basis for material partitioning in numerical simulations. The extracted geological surfaces can be spatially aligned with the DEM data and sorted according to elevation, as the circles shown in Fig. 8a. During the structured particle filling process along the z -axis, each points (triangles in Fig. 8b) is compared with these surfaces based on its vertical position, thereby determining its associated material label. When the filling mode—a strategy described in Section 3.4—is activated, each structural point (e.g., circles or triangles) is surrounded by four replicated points, or a user-specified number. Consequently, the colored regions in Fig. 8b illustrate the effective influence domain of each property partition.

For structural zoning, encompassing both natural and engineered structures, irregular and complex geometries are often present. In such cases, we recommend using established modeling software to construct each structural component, which can then be exported as separate STL surface models. Each component is subsequently discretized into structured material particles following the algorithms described in Section 3.1. Finally, all particles are aligned within a common background grid, and duplicate particles are removed to ensure consistency and eliminate redundancy. Given the high efficiency of our particle generation framework, this repeated procedure incurs negligible computational overhead.

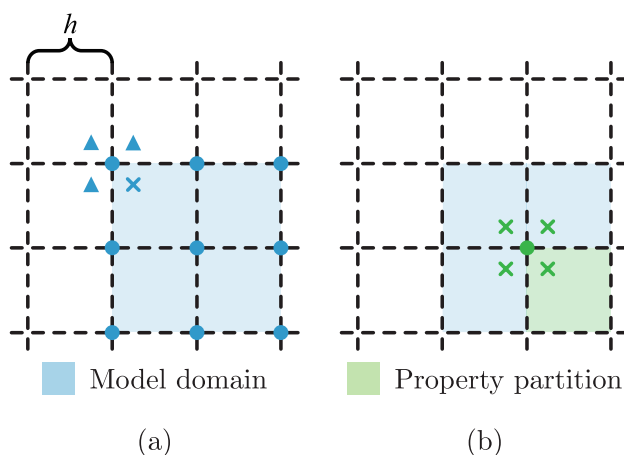


Fig. 9 Geometric (a) and property assignment (b) errors introduced by the filling mode

3.4 Evaluation

The evaluation of the proposed structured particle generation framework focuses on two aspects: particle quality and generation efficiency. The quality assessment examines whether particles are correctly distributed within the target domain, and the accuracy of physical property assignment to each particle. Let the target geometry be denoted as $\mathcal{G}$, and the set of structured particles as p_i . If a particle $p_i \notin \mathcal{G}$, it is classified as invalid. The geometric error is then calculated as the percentage of such invalid particle relative to the total number of particles in the test set. It is generally difficult to derive the geometric error directly from the discretization algorithm. A common practice is to check each point individually for geometric validity. However, estimating the worst-case geometric error based on algorithm characteristics is both meaningful and feasible. This helps reveal error sources and guides the selection of an appropriate grid spacing h . Similarly, we estimate the maximum property error under a fixed number of three property partitions. In terms of performance, we evaluate the generation speed across a variety of model types, with particular attention to high-resolution scenarios. Given that the framework supports both general-purpose geometric models represented by STL files and terrain models encoded as DEM data, we use these as ground truth to comprehensively assess the framework's applicability and robustness across diverse use cases.

The structured material particles obtained from the discretization methods described in Sections 3.1 and 3.2 are not the final simulation particles, but rather serve as the center points of activated cells in the background grid used for MPM simulations. After assigning appropriate physical attributes (labels) to these center points, each is expanded into four particles per cell in 2D (eight in 3D) by replicating the attribute information—an operation we refer to as the “filling mode”. The number of particles per cell can be adjusted flexibly, including anisotropic configurations where different numbers of particles are assigned along each spatial direction. This approach is made possible by the design of the framework, which consistently generates center points on a reference background grid with a specified resolution h , identical to the cell size used in the MPM background grid. As a result, all particles produced by the framework lead to uniform counts per cell at the start of the simulation. Ignoring numerical precision errors and with filling mode disabled, all discretization algorithms in our framework—except for voxelization—do not introduce geometric errors, and thus no property errors occur. Therefore, our analysis is conducted under the condition that filling mode is enabled, while the errors introduced by the voxelization algorithm are separately evaluated and discussed in Section 4.2.

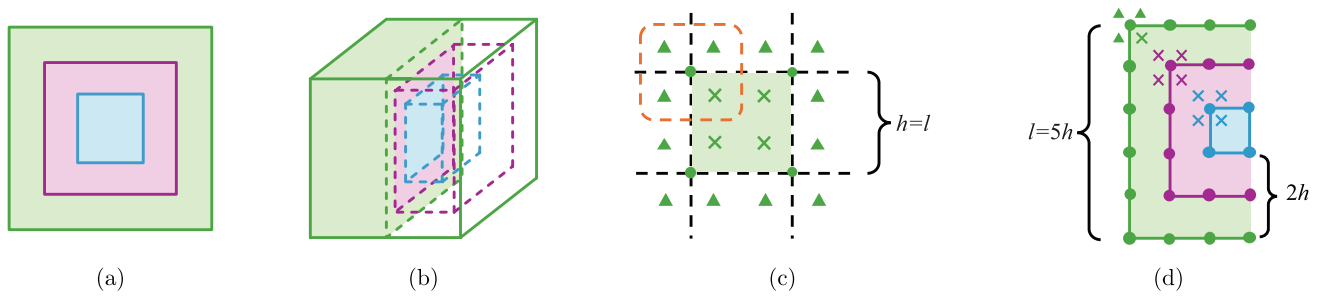


Fig. 10 Configurations of the test models in 2D (a) and 3D (b). c Maximum 2D geometric errors: all triangles within the red dashed box represent incorrectly placed particles generated under the filling mode.

d Maximum 2D property assignment errors: mismatches between the colors of markers (triangles and crosses) and their respective background regions indicate labeling errors

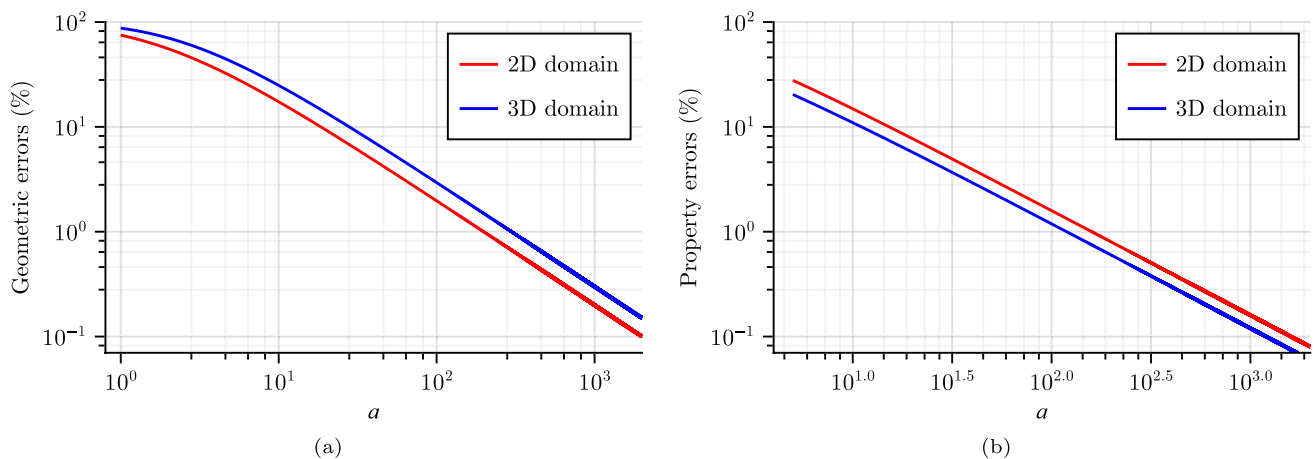


Fig. 11 Geometric (a) and property assignment (b) errors, where a is a dimensionless parameter representing the ratio between the model side length and the background grid spacing

Prior work by Perez et al. [22] demonstrated that the initial spatial distribution of material particles can affect simulation accuracy under pre-stressed conditions. To mitigate this source of error, the filling mode ensures regular and consistent placement of particles at initialization. This design also offers the following advantages: (i) for a 2D model with an initial number of structured particles n_p , uniform filling increases the total particle count to $4 \times n_p$, effectively increasing the particle generation speed by a factor of four without additional cost; and (ii) the framework naturally supports local mesh refinement by adjusting the number of filled particles per cell. This filling process is optional, preserving the generality and flexibility of the framework.

3.4.1 Accuracy verification

As illustrated in Fig. 9a, the circles represent the center points discretized by the proposed framework, with a background grid spacing of h . Prior to applying the filling mode, all center points conform to the geometric boundaries of the model. However, after filling, in the most extreme case, up to three of the four generated particles may fall outside the

domain, with only the blue cross remaining inside the valid region. While interior particles do not violate geometric discretization, they may still result in incorrect property assignments. As shown in Fig. 9b, the green point, initially located within a green-labeled region, is expanded into four particles with inherited properties during filling. Ideally, only the lower-right cross should retain the green property, while the other three should be assigned the blue label, reflecting their true spatial region. Based on these observations, we construct stress-test cases in both 2D and 3D cases to quantify the maximum possible error introduced by the filling mode. The model configurations are shown in Fig. 10.

Taking the 2D square case shown in Fig. 10a as an example, the model domain is defined as the region enclosed by the green square, where different colors represent distinct material property partitions. Let the side length of the square be l , and the grid resolution used in simulation be h . When $l/h = 1$ and the square is perfectly aligned with the grid, the worst-case error scenario occurs, as illustrated in Fig. 10c. Note that l/h must not be less than 1, since this would imply no discretization is needed. In this extreme case, 16 particles are generated and 75% geometrically incorrect as they lie outside the model (green triangles) region.

Based on this analysis, we define the dimensionless parameter $a = l/h$, and derive analytical expressions for the geometric error rate under this configuration, which are given by

$$\varepsilon_{g,max} = \begin{cases} 1 - \left(\frac{a}{a+1}\right)^2, & \text{for 2D} \\ 1 - \left(\frac{a}{a+1}\right)^3, & \text{for 3D} \end{cases} \quad (22)$$

Equation (22) can be used to guide the selection of the grid resolution h . For example, in the extreme 2D case, achieving a geometric error rate below 2% under the filling mode requires that $a \geq 100$ (Fig. 11a). It is worth noting that the geometric error rate reflects particles located outside the actual geometry, their distances from the true model boundary remain

small: no greater than $\sqrt{2}h/4$ in 2D and $\sqrt{3}h/4$ in 3D, as these particles are situated near corner regions.

To evaluate the property assignment error rate, we modify the previous geometric error test case by introducing two additional property regions within the model interior, resulting in a total of three distinct partitions. This configuration still represents a worst-case scenario in terms of potential error (Fig. 10d). Although the outermost layer of particles exhibits geometric errors, their property assignments remain correct and are therefore excluded from this analysis. Based on this setup, the analytical expressions for the property assignment error rate are given by

$$\varepsilon_{p,max} = \begin{cases} \frac{8a+10}{5(a+1)^2}, & \text{for 2D} \\ \frac{6a^2+12a+10}{5(a+1)^3}, & \text{for 3D} \end{cases} \quad (23)$$

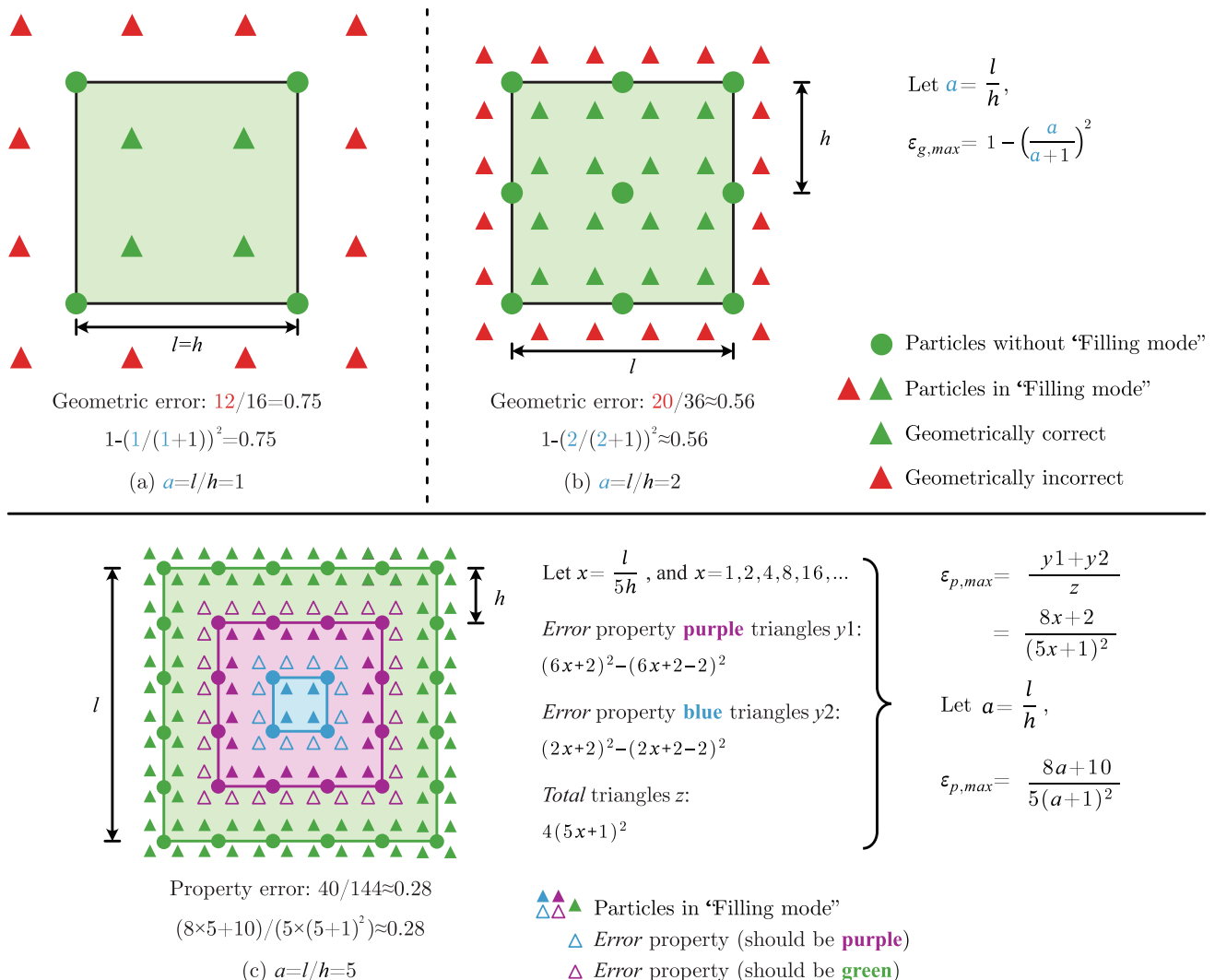


Fig. 12 Schematic derivation of the equations for the 2D maximum geometric error and the maximum property error in the case of three-part partitioning. **a**, **b** illustrate the patterns of maximum geometric

error for different values of a , while **c** presents the calculation method for the property error, where $a \in \{5 \cdot 2^n \mid n = 0, 1, 2, \dots\}$

Similar to geometric error rates, these points assigned with incorrect attributes are still located very close to the partition boundary.

Equations (22) and (23) are inferred from the error behavior observed under extreme (worst-case) scenarios. Using a two-dimensional model as an example, Fig. 12 presents how the maxima of the geometric and property errors vary with the parameter a . It should be emphasized that blocks are chosen as test objects because, for the same characteristic size, they attain the error upper bound, rather than because their geometry is convenient for derivation or computation.

As shown in Eqs. (22) and (23), both the geometric and property errors of the discretized particle distribution are directly related to the background grid resolution h , assuming a fixed model size. When h is appropriately chosen, the resulting errors have negligible impact on MPM simulation accuracy. Moreover, the derived expressions represent worst-case scenarios; in practice, a small offset is applied to the reference grid to avoid exact alignment with the model geometry. In simulations that are particularly sensitive to particle placement or material properties, the filling mode can simply be disabled to eliminate such errors.

3.4.2 Efficiency verification

The particle discretization process must be both efficient and resolution-adaptive to support a wide range of simulation requirements. Ideally, this discretization should be significantly faster than the construction of the STL model itself. Naive methods, such as ray casting with the Even-Odd rule, exhibit a computational complexity of $\mathcal{O}(n \times m)$ where n is the number of query particles and m is the number of triangle faces. This becomes computationally prohibitive when both n and m reach the scale of millions. In contrast, the two methods introduced in Sections 3.1.2 and 3.1.3 offer much simpler implementations and avoid the need for complex spatial structures such as BVH.

Since structured particle generation is part of the pre-processing pipeline for MPM simulations, real-time performance is not required. From the outset, our framework is designed to ensure all algorithms are efficient on CPU platforms alone. Performance evaluation focused primarily on

scenarios involving high-resolution input models. During testing, we observed that our framework consistently completed the generation of millions of particles within seconds. Moreover, the performance remained stable across different CPU configurations. All subsequent performance evaluations were conducted using Julia v1.11.5 on a laptop equipped with an Apple M4 Pro CPU and 22.35 GiB of RAM.

4 Results and discussion

We first consider a benchmark case featuring a 2D landslide cross-section, a data type that is ubiquitous in geotechnical applications of MPM. Structured particles are generated at several resolutions, and zone-based material properties are assigned to the particles accordingly. The framework is then tested on two more complex 3D geometries. The “Dragon–Phoenix” model, with its intricate surface ornamentation, requires a very fine discretization to reproduce the original geometry faithfully. The “Wheel” model is further complicated by slender spokes; a similarly high resolution is needed to resolve both the spokes and the tire tread. Finally, we evaluate on complex terrain models typical of engineering practice, including stratified models constructed from geological stratification surfaces and models discretized along failure surfaces obtained with the SLBL method. For every case study, filling mode is activated. We examine and discuss (i) the geometric errors as a function of resolution, (ii) the property errors in the relevant examples, and (iii) particle generation speed.

4.1 2D landslide profile

As illustrated in Fig. 13, we adopt the landslide cross-section introduced by Yin et al. [37] to represent the underlying geological structure. The cross-section depicts the geological and landfill conditions prior to and following the landslide. The profile is discretized into structured particles, and material properties are assigned to individual particles. Five properties are specified: (i) alluvial fan, (ii) granite bedrock, (iii) waste accumulation, (iv) undisplaced waste, and (v) displaced waste.

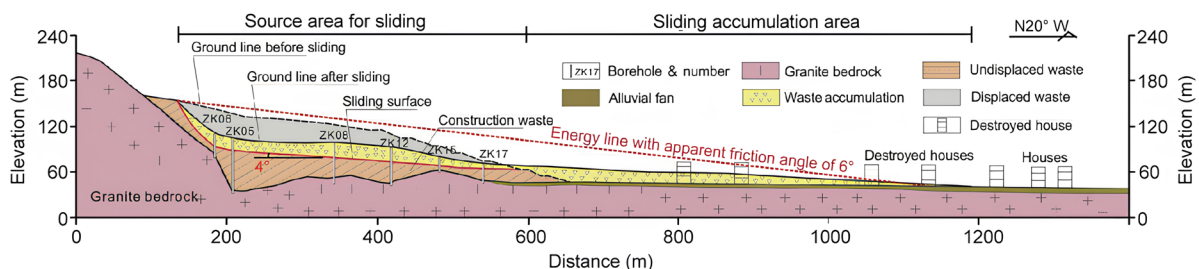


Fig. 13 Landslide profile used to test the 2D structured particle generation (reproduced from Yin et al. [37])

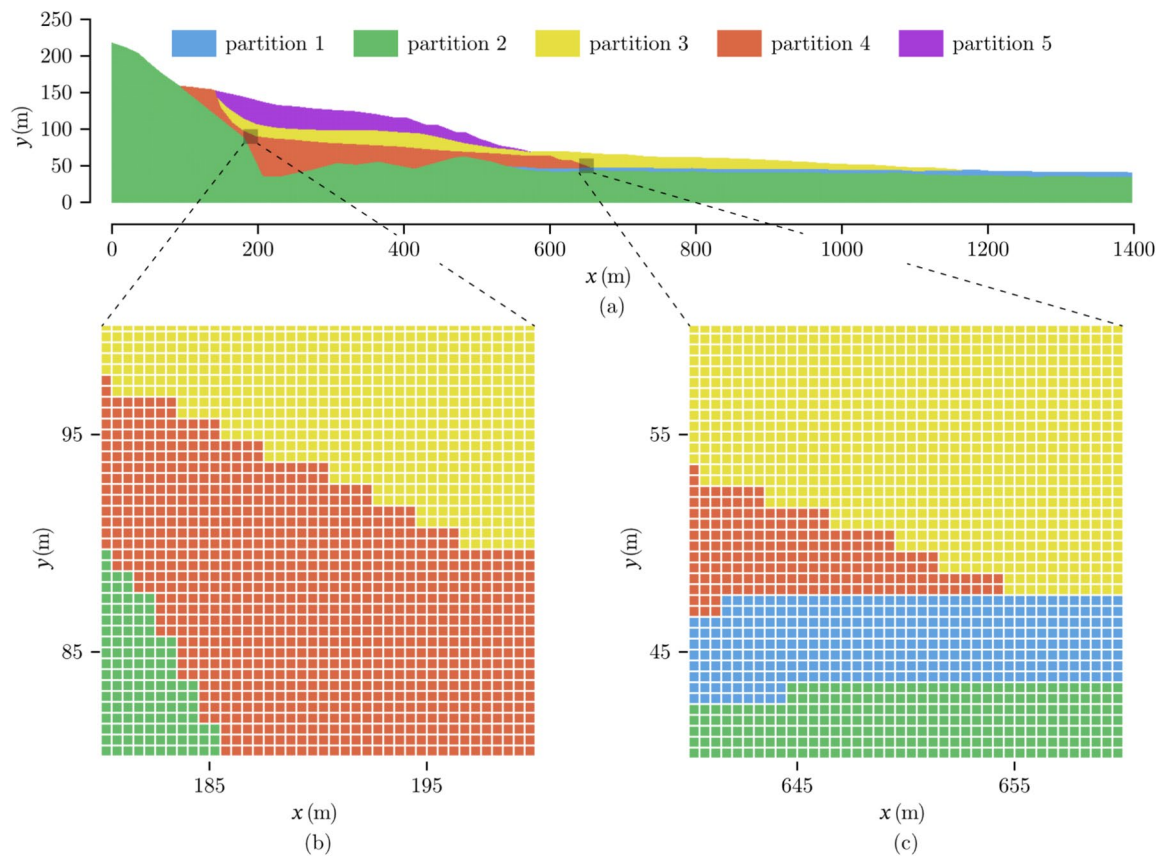


Fig. 14 2D landslide discretization result

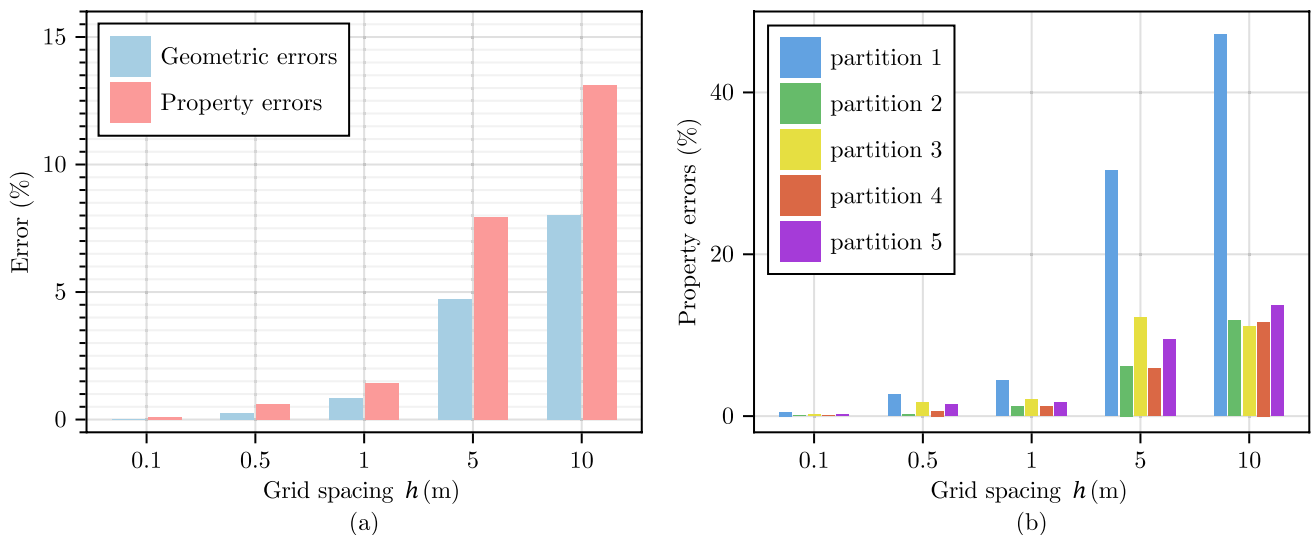


Fig. 15 Geometric and property errors in the 2D testing. **a** Shows the ratio of incorrectly particles to the total number of particles, while **b** presents the ratio of particles with incorrect property assignments to the total number of particles within each partition

The profile was discretized with the procedure detailed in Section 3, and the resulting particle layout is presented in Fig. 14. Coordinates of every internal property interface, as well as the outer boundary, are digitized from Fig. 13. The small inaccuracies introduced by this digitization

originate from the sketch itself rather than from the particle-generation algorithm, so the extracted data are treated as the ground truth for the subsequent error analysis. We first investigated two error-prone zones (the square patches highlighted in Fig. 14a). Magnified views show that no material

Table 1 Performance results for the 2D test (excluding I/O time)

Grid spacing h	Number of particle	Execution time (s)
10	4880	–
5	19,720	–
1	480,216	–
0.5	1,909,776	0.04
0.1	47,739,672	0.28

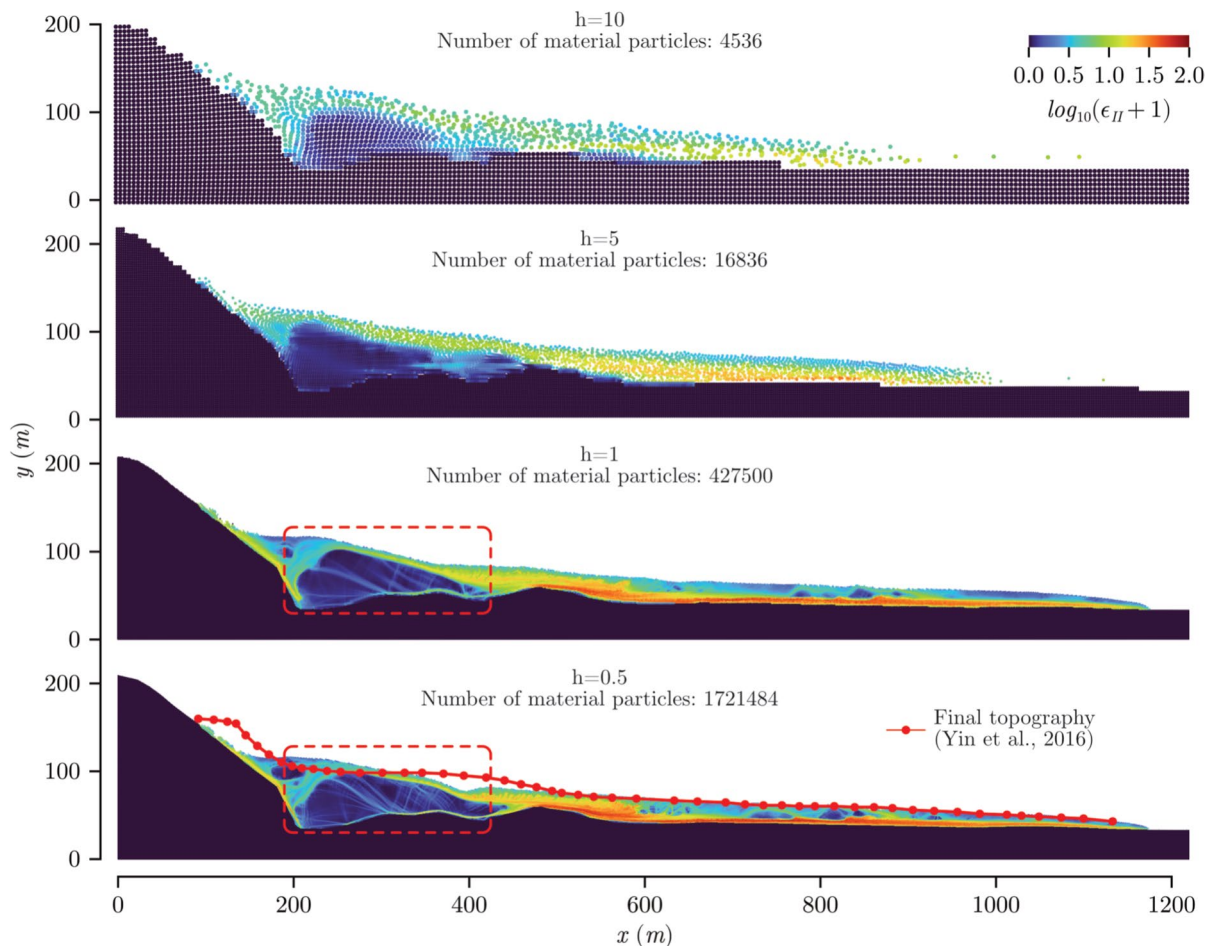
The first three execution times are ignored due to short computation time

interpenetration occurs near the interfaces. One exception demands attention: the blue layer in Fig. 14c is so thin that it may disappear entirely if the resolution (i.e., grid spacing h) is set too coarse.

Figure 15a shows geometric and property errors across various resolutions, defined as the percentage of incorrectly assigned particles relative to the total particle count. Both errors decline with increasing resolution. Figure 15b presents property errors for individual partitions at different resolutions, calculated as the ratio of erroneous particles to the total particles within each partition. Due to the geometric characteristics of partition 1, particularly its thinness relative to the overall model, it exhibits the highest property error.

Table 1 summarizes the performance benchmark results of the particle generator. The model achieves its highest resolution at $h = 0.1$, generating 47,739,672 particles. For a 2D model, this particle count indicates a high-resolution scenario. Remarkably, the discretization step takes only 0.28 s, meaning file I/O operations significantly surpass computation time. Consequently, our framework presents no performance bottleneck for 2D models.

To emphasize the influence of model resolution on simulation outcomes, MPM simulations are conducted for this case. The computational domain corresponds to the solid-line region in Fig. 13, rather than the area shown in Fig. 14. Consequently, the number of material particles under the same grid spacing differs from the values presented in Table 1. The simulation adopts a Drucker–Prager constitutive model with strain-softening behavior, described by the exponential law $x = x_r + (x_p - x_r)e^{-\eta\epsilon_{II}}$, where x represents either the friction angle ϕ or cohesion c ; x_p and x_r are the peak and residual values, respectively; ϵ_{II} is the equivalent plastic strain; and $\eta = 400$ is the softening parameter. The bedrock is modeled with a density of 2700 kg/m³, Young’s modulus of 50 GPa, and Poisson’s ratio of 0.25, while the soil is assigned a density

**Fig. 16** MPM simulation results at four different resolutions (grid spacing h)

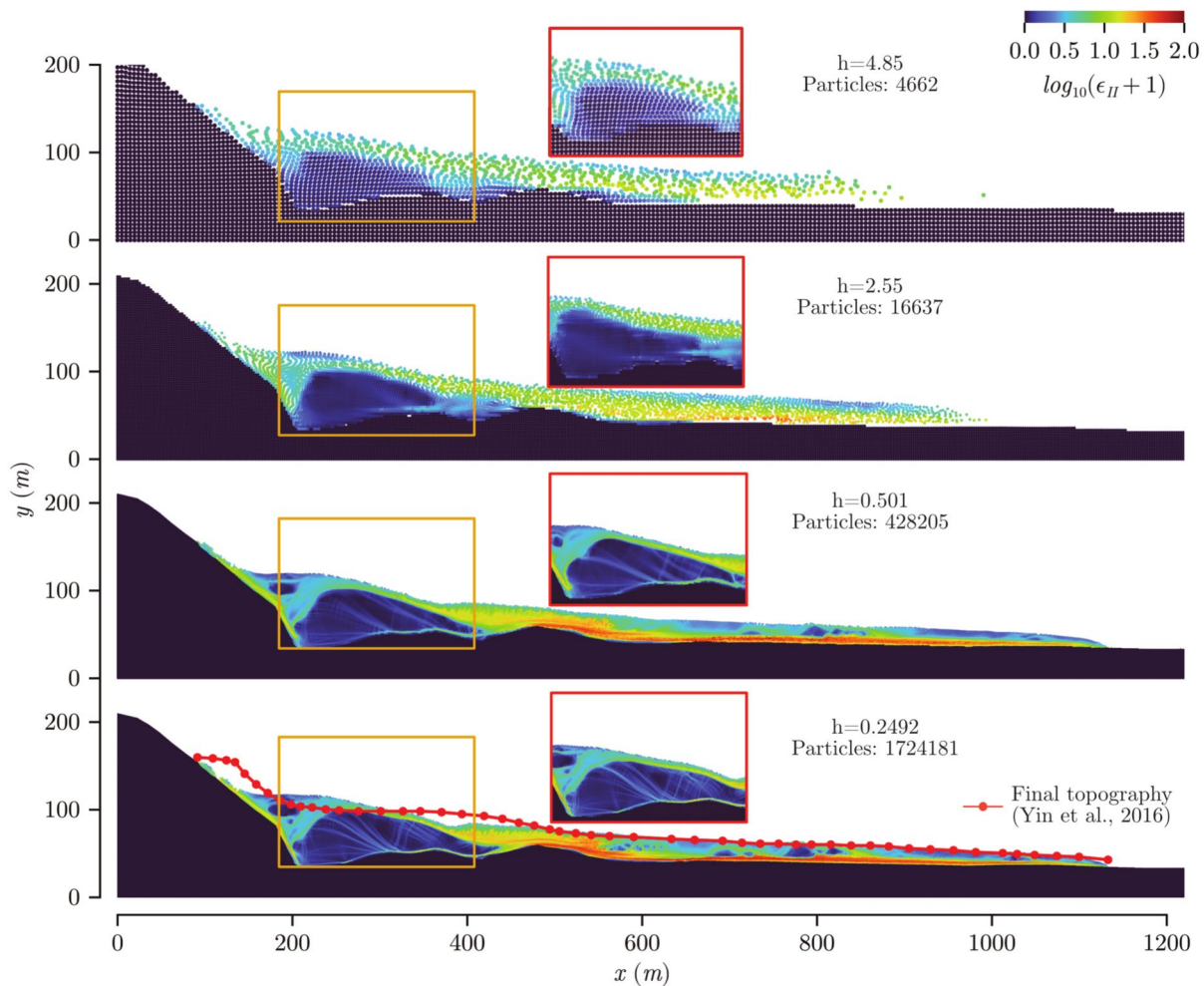


Fig. 17 Effect of geometric errors on landslide simulation: results with geometric errors (red windows) and without geometric errors show nearly identical shear bands at all four resolutions

Table 2 Performance results for the 2D test (excluding I/O time)

Grid spacing h	Number of particle	Execution time (s)
4.85	4662	0.16
2.55	16,637	0.18
0.501	428,205	0.52
0.2492	1,724,181	1.49

Since the filling mode was disabled, no geometric error was introduced. Particle numbers should be compared with Fig. 16, not Table 1, as the discretized models in Table 1 are not those used in the MPM simulations

of 1800 kg/m^3 , Young's modulus of 30 MPa , and Poisson's ratio of 0.26 . The peak friction angle of 24.8° , residual friction angle of 1° , peak cohesion of 22 kPa , and residual cohesion of 18 kPa . Further details of the model configuration can be found in the work of Shi et al. [34].

We conducted simulations using **MaterialPointSolver.jl**, a high-performance, backend-agnostic MPM solver [14], at four different spatial resolutions. The results are shown in Fig. 16. At low resolutions (e.g., $h = 5, 10$), the run-out

distance exhibits significant variation with changing h , but begins to stabilize when $h \leq 1$. At higher resolutions (e.g., $h = 1, 0.5$), well-defined shear bands emerge, as indicated by the red dashed rectangular regions. The development of these localized weak zones governs the final topographic configuration and provides critical insight into landslide potential. Such features cannot be resolved in low-resolution models.

A comparison between the MPM simulation results and the final deposition topography (Fig. 16, bottom) demonstrates excellent agreement. When the filling mode is activated, geometric errors arise from particles generated outside the model boundary. To evaluate their impact, we disabled the filling mode and built four error-free models with particle counts comparable to those in Fig. 17. The performance benchmark results are summarized in Table 2. The comparison shows that: (i) the geometric error has negligible influence on the landslide simulation at both low and high resolutions; and (ii) the geometric error decreases as the resolution increases (see Fig. 15). Hence, the filling

Table 3 Total I/O time for reading the STL model and writing particle data under different grid spacings

Model	Grid spacing h (mm) $\rightarrow$ I/O time (s)				
Dragon–Phoenix	0.3 $\rightarrow$ 11.8	0.5 $\rightarrow$ 10.17	1 $\rightarrow$ 1.37	5 $\rightarrow$ 2.63	10 $\rightarrow$ 2.58
Wheel	1.0 $\rightarrow$ 28.08	1.2 $\rightarrow$ 17.46	2 $\rightarrow$ 11.84	3 $\rightarrow$ 10.53	5 $\rightarrow$ 9.91

mode significantly improves particle generation efficiency without compromising simulation accuracy.

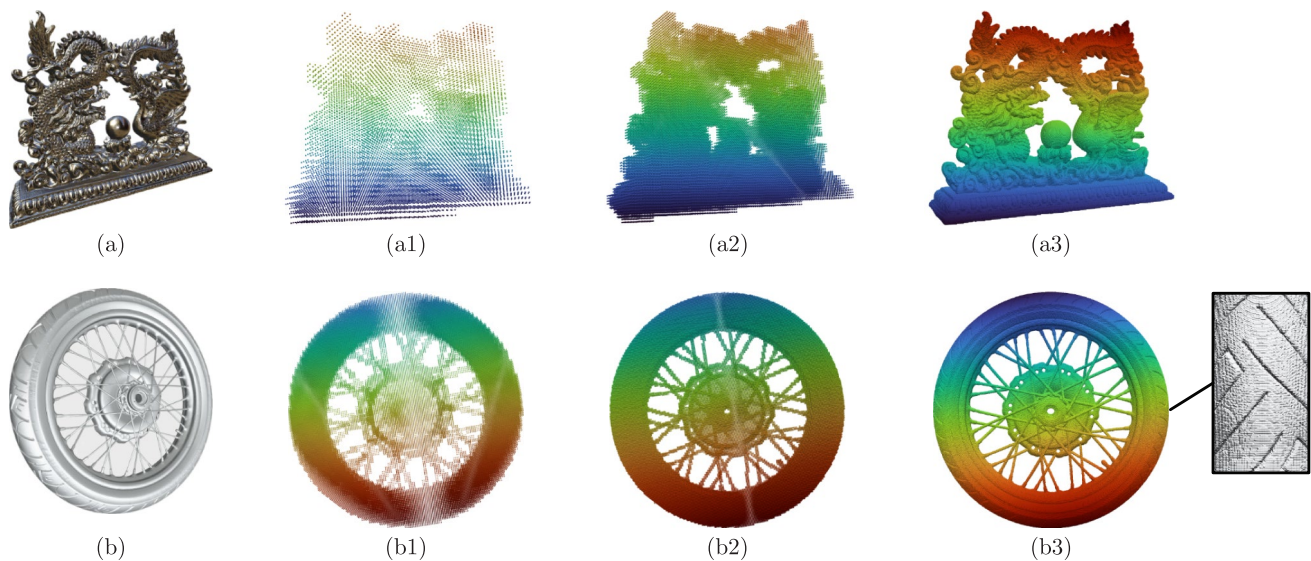
This case highlights the necessity of using high-resolution models and emphasizes the critical role of both accuracy and efficiency during model preprocessing. The proposed framework enables the rapid generation of tens of millions of structured material particles within a few seconds, making it highly effective for discretizing models at varying levels of resolution. It also demonstrates that, although the filling mode introduces an h -dependent error, it

has negligible influence on the landslide simulations while significantly improving the discretization efficiency.

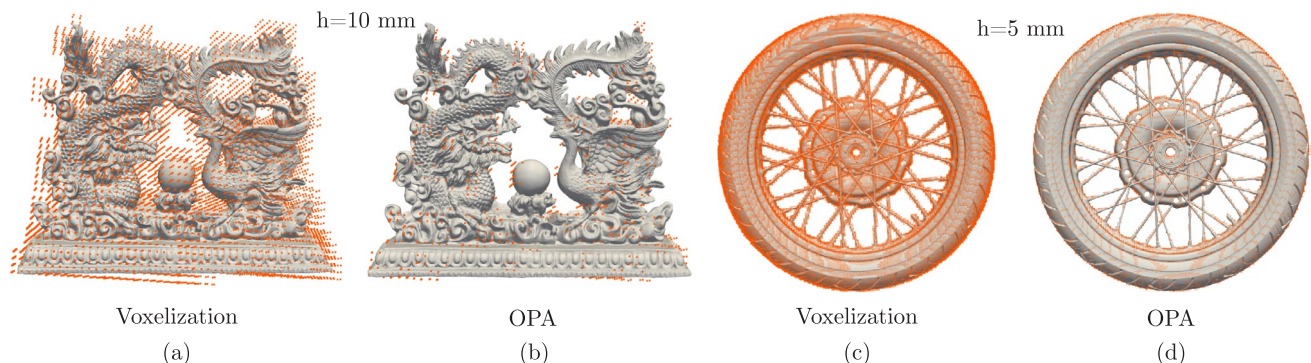
4.2 3D models

We evaluate our framework using two complex models: the “Dragon–Phoenix” and the “Wheel”. The Dragon–Phoenix model contains approximately one million triangular faces, while the Wheel model comprises around three million. Such geometric complexity presents a significant challenge. File I/O operations, including reading the STL files and writing the resulting XYZ files, are recorded separately in Table 3, as they take substantially longer than the particle generation itself.

Both algorithms introduced in Sections 3.1.2 and 3.1.3 were implemented and evaluated. Figure 18 summarizes the results: the top row reports the Dragon–Phoenix model, and the bottom row the Wheel model. Subfigures (a1–3) and (b1–3) show

**Fig. 18** Structured particle discretization results of two 3D models. **a** Dragon–Phoenix model (size: 220×68×175 mm); **b** Wheel model (size: 660×117×660 mm). Subfigures **a1–3** and **b1–3** correspond to visualiza-

tions under different grid spacings h . In **b3**, the tread pattern on the wheel surface is clearly visible.

**Fig. 19** Comparison of surface geometric errors between voxelization and the optimized projection-based algorithm (OPA) under identical grid spacing. In both models, voxelization results in a greater number

of inaccurate surface samples (orange points), whereas the optimized projection-based algorithm yields higher geometric fidelity and generates fewer structured particles

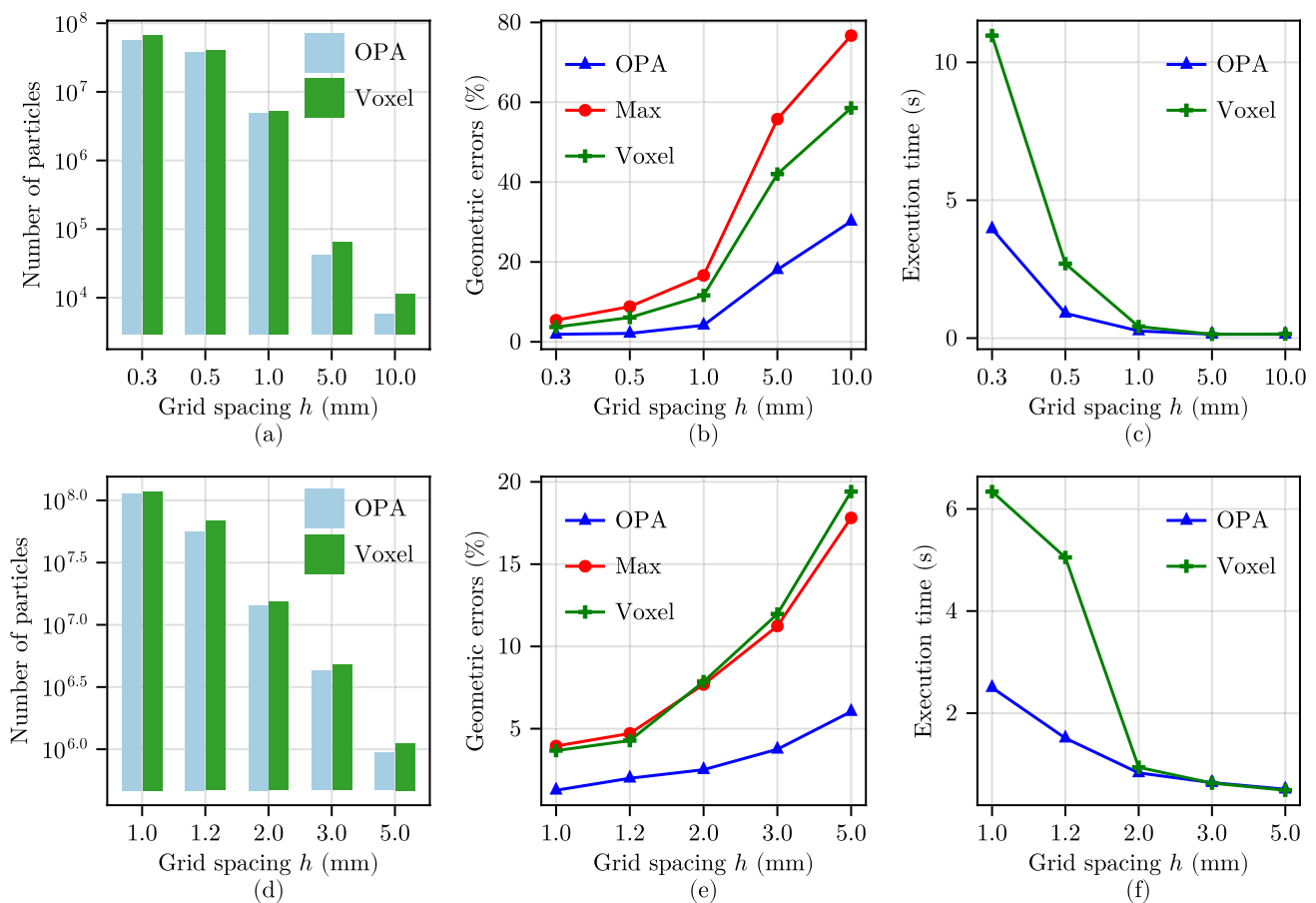


Fig. 20 Evaluation of geometric errors and performance for two 3D model discretization methods. The first and second rows correspond to the Dragon–Phoenix and Wheel models, respectively

the number of structured particles generated at different resolutions. For the Wheel model we reached 1×10^8 particles—a resolution seldom attempted in MPM simulations.

Across all tests the voxelization strategy produced more particles than the optimized projection-based algorithm, and its geometric error is consistently larger (Figs. 20b, e). The discrepancy arises because voxelization first samples points on the triangular facets of the STL mesh. Each sample is then snapped according to the grid spacing h ; when the grid and triangle do not coincide, the sample may be displaced outside the facet. In filling mode every misplaced point is replicated to the eight neighboring particles, compounding the error.

To highlight this behavior we superimposed the lowest-resolution particle set on the original STL surface in Fig. 19, which shows that the optimized projection-based algorithm adheres much more closely to the geometry. The error bound derived in Section 3.4 applies only to the optimized projection-based algorithm and does not account for the pre-filling misallocations introduced by voxelization. Consequently, voxelization always yields a larger particle count in Figs. 20a, c, and an error that exceeds the theoretical maximum in Fig. 20e.

Finally, Fig. 20c, f compare the execution time of the two methods. For coarse resolutions (large h) their performance is nearly identical, but as h decreases the voxelization method becomes slower than the optimized projection-based algorithm.

The slowdown is attributable to the fact that, when the grid spacing decreases but Δ^* —which controls the sampling density—remains fixed, the sampling becomes too sparse relative to the finer grid. As a result, many surface-adjacent grid nodes are not marked by any sampling point, increasing the cost of completing a closed voxel boundary. Even so, both methods complete in under 10 s for every model, which is comparable to or even shorter than the time required to load the largest STL file.

4.3 Complex terrain

We first test a case where DEM data is processed under the assumption that geological stratification information is known in advance, referred to as Scenario 1 (see Fig. 21). The entire model is divided into four zones. All stratification surfaces, as well as the DEM terrain surface, are rasterized

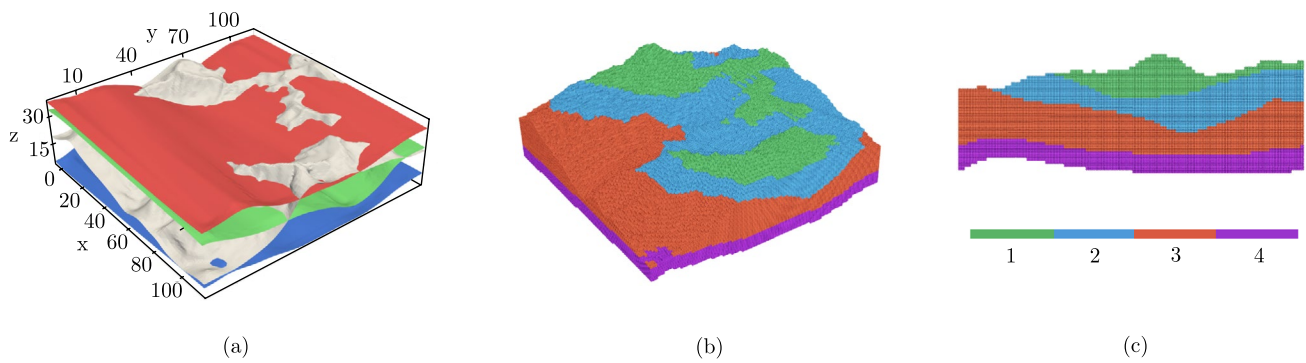


Fig. 21 Particle discretization results for 3D complex terrain (Scenario 1). **a** The model is divided into four distinct zones using geological structural surfaces in different colors; **b**, **c** depict the discretization results and the particle distribution along the cross-section, respectively

Fig. 22 Discretization results of the model for failure surface prediction using the SLBL method (Scenario 2). **b** is a magnified view of the white dashed region in **a**. The purple base surface in **b** represents the failure surface obtained via the SLBL method. Structured particles are generated between the original DEM surface (green) and the failure surface (purple), and are assigned to the *Slide* region, while the purple area is designated as the *Base* in the MPM simulation

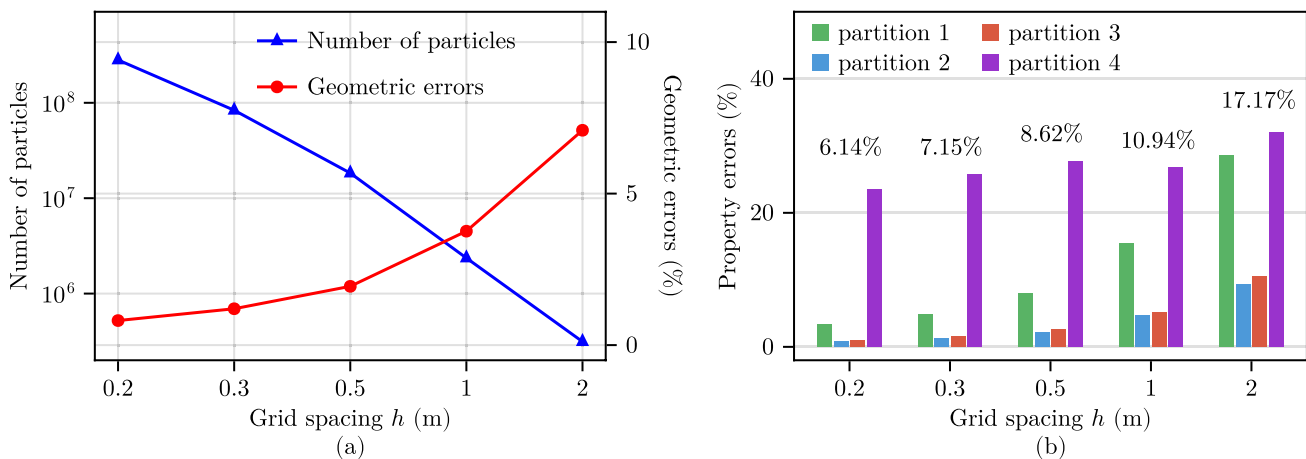
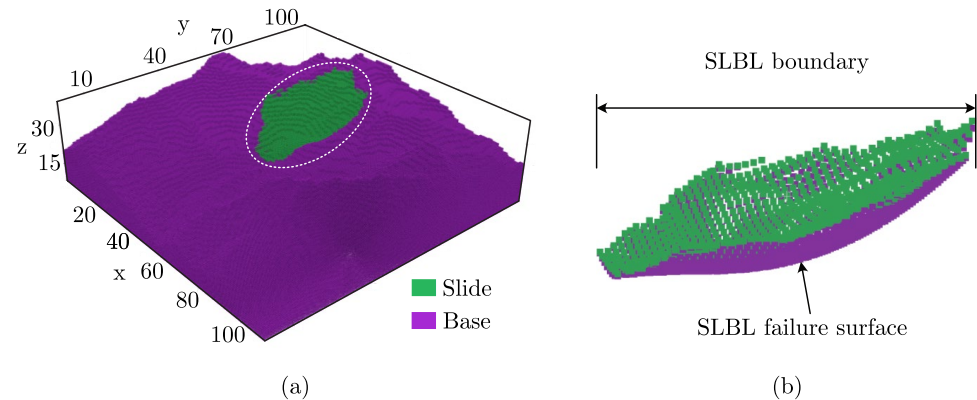


Fig. 23 The geometric and property errors in complex terrain model (Scenario 1)

in the xy -plane using the method described in Section 3.2. Each stratification surface can be defined within a custom 2D domain (not necessarily a rectangular region), as long as alignment is maintained. In this scenario, the bottom surface of the model is not flat but defined by a user-specified surface. The property layers serve illustrative purposes only and do not represent physically meaningful strata.

Scenario 2 is defined as an MPM model constructed from surface DEM data by predicting the failure surface using

the SLBL method (see Fig. 22). Once the SLBL boundary is determined, two parameters, $Z_{\max}$ and L_{rh} , must be specified (see Appendix A for definitions). A comparison between the predicted failure surface and the original DEM is shown in Fig. 22b. After obtaining the failure surface, the entire model can be discretized directly; alternatively, only the sliding zone may be discretized, with the base zone defined as a frictional interface in the MPM simulation, as illustrated in the supplementary material.

Table 4 Performance benchmark results for the two scenarios (excluding I/O time)

Scenario	Grid spacing h				
	0.2	0.3	0.5	1	2
1	4.80 s	1.24 s	0.27 s	0.03 s	–
2	4.37 s	1.19 s	0.25 s	0.03 s	–

It is also possible to generate structured particles from STL surface models derived from DEM data using voxelization or the optimized projection-based algorithm. However, since most DEM data are represented as point clouds, we focus our evaluation on the method discussed in Section 3.2. As illustrated in Fig. 23a, both scenarios produce the same number of structured particles with a maximum geometric error of less than 10%. Our primary focus is on the stratigraphic property assignment in Scenario 1, as shown in Fig. 23b. For each grid spacing, the property errors within each partition is represented by bar charts, along with the overall errors across the entire model. Notably, partition 4 exhibits a higher error rate due to its location at the bottom of the model—being the thinnest layer and occupying the entire xy -plane. This behavior is similar to that observed in Section 4.1 (see Fig. 15b, partition 1).

The performance evaluation is presented in Table 4. The overhead of the SLBL method is minimal, with computation time that is negligible. Scenario 2 effectively involves assigning two properties to the model (compared to four in Scenario 1), yet its execution time is nearly identical to that of Scenario 1. This indicates that the property assignment in our framework is also not a performance bottleneck.

4.4 Model setup for parametric landslide study

To systematically evaluate the influence of geometric (failure surface) and material uncertainties on potential landslide failure patterns and run-out behavior, we employ a particle generator integrated with the SLBL method, thereby establishing a reproducible, configuration-driven preprocessing workflow for MPM simulations and enabling a focused parametric investigation. A three-dimensional landslide model with simple geometry (Fig. 24) is adopted, and a two-stage analysis is designed to examine (i) geometric uncertainty and (ii) material uncertainty. In Stage 1, variations of the SLBL parameter ($Z_{\max}$, as explained in Appendix A) is performed to obtain a set of plausible failure surfaces and to quantify the corresponding sliding volumes. In Stage 2, a representative failure surface is selected and the domain is divided into four material partitions, each characterized by inherent physical parameters (Fig. 25). Controlled perturbations are applied to the thickness of each partition to evaluate the sensitivity of the predicted maximum run-out distance. Apart from the explicitly varied factors, all numerical settings—including particle and grid discretization, boundary and initial conditions, and solver controls—are kept identical across stages to ensure comparability and reproducibility.

The SLBL boundary in the xy -plane has a major axis of 27.3 m and a minor axis of 15.7 m. Accordingly, L_{rh} (as explained in Appendix A) is set to 27.3 m, and four values of $Z_{\max} = (3, 7, 11, 15)$ m are employed to predict the

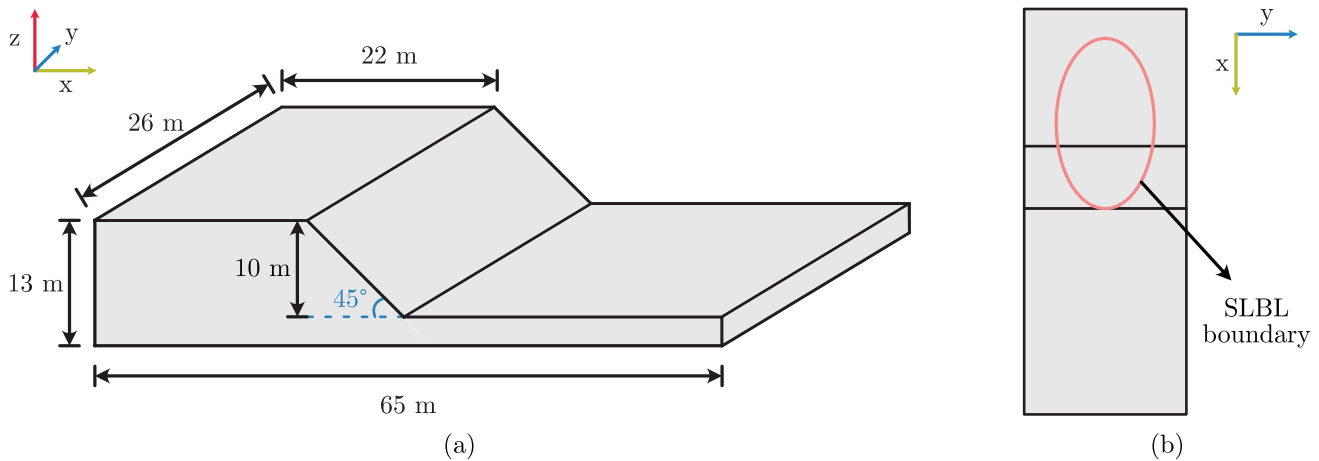
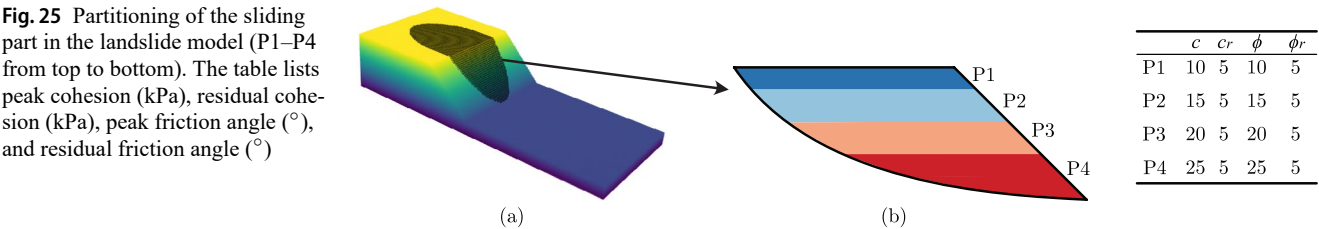


Fig. 24 Dimensions of the landslide model (a) and prescribed boundary for the SLBL method (b)



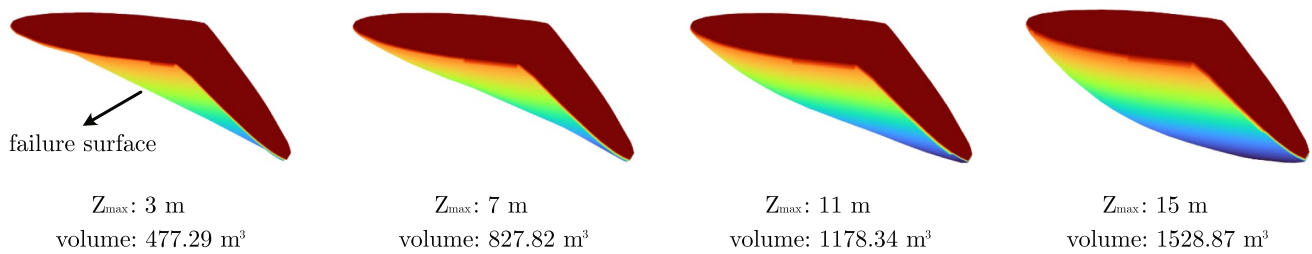
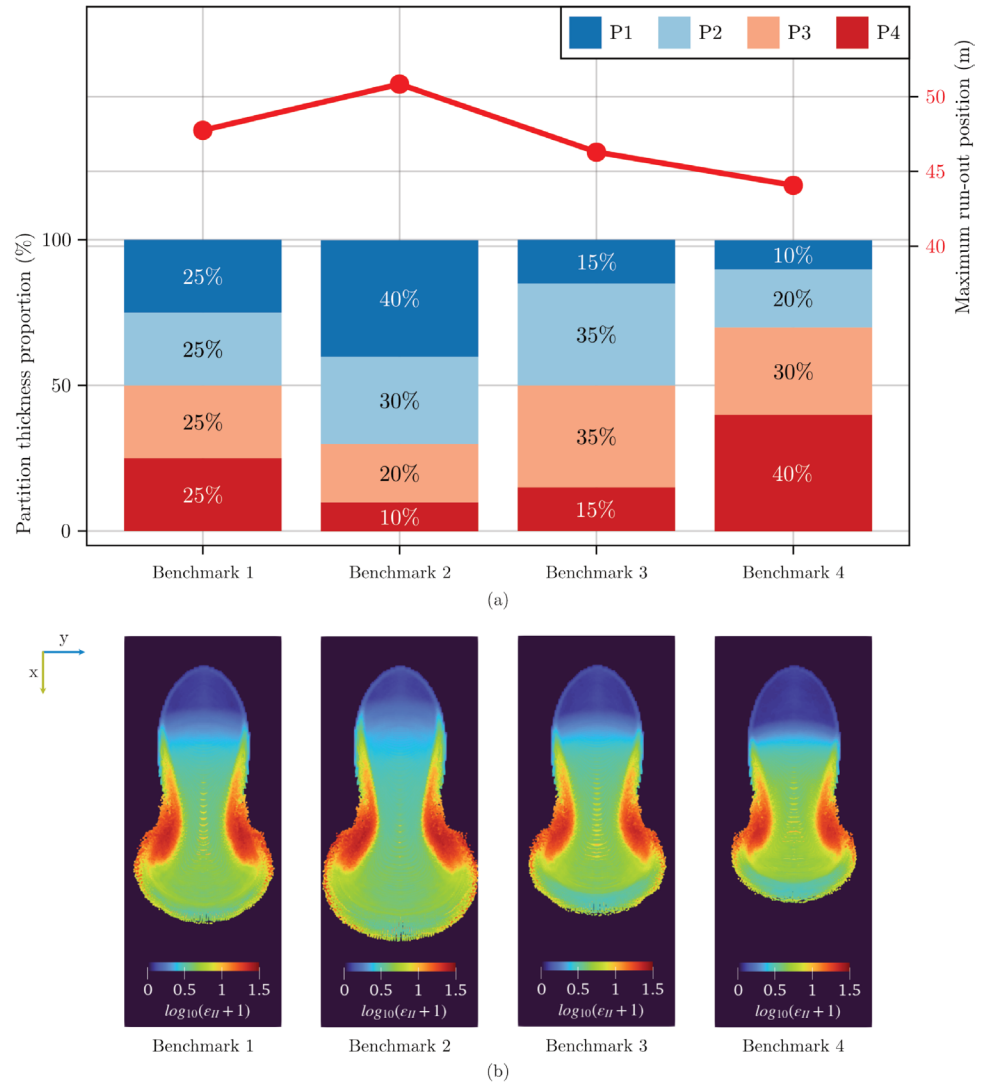


Fig. 26 Predicted sliding part for different values of $Z_{\max} = 3, 7, 11, 15$ m, showing the corresponding volumes and geometries

Fig. 27 a Partition thickness ratios for four benchmarks: (i) equal-thickness, (ii) upper-dominated, (iii) middle-dominated, and (iv) lower-dominated; the maximum run-out is indicated. **b** Plan-view plastic strain at the end of the simulations, showing the sensitivity of run-out to partition thickness



sliding surface. As shown in Fig. 26, the volumes between the predicted sliding surface and the upper surface (hereafter referred to as the sliding part) are obtained for different $Z_{\max}$ values. The sliding surface derived with $Z_{\max} = 15$ m provides the closest approximation to the actual failure geometry and is therefore selected for subsequent analyses.

The computational domain is discretized with a grid resolution of $h = 0.3$ m, with eight particles uniformly distributed per cell, yielding a total of 3,678,952 particles. The sliding part is

modeled using the Drucker–Prager constitutive law, while the remaining domain is treated as elastic. The sliding part is further subdivided into four partitions (P1, P2, P3, and P4), with the relative thickness of each partition introduced as a control variable. The total simulated physical time is 15 s, of which the first 8 s correspond to the elastic loading phase. Based on different thickness ratios of the four partitions, four benchmark simulations are conducted to assess the sensitivity of the maximum predicted run-out position (x-coordinate) at the end of the simulation.

Fig. 28 Partitioning of Benchmark 5. The material properties of Partitions 1–4 are specified as shown in Fig. 25, while Partition 5 shares the same properties as Partition 1. The thickness fractions of Partitions 1–4 are identical

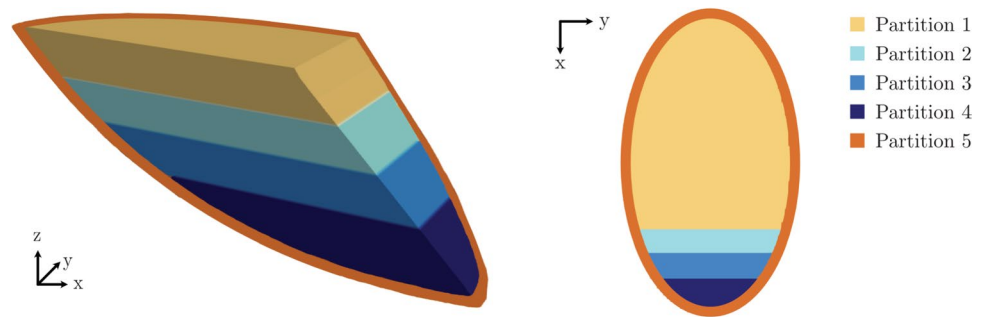
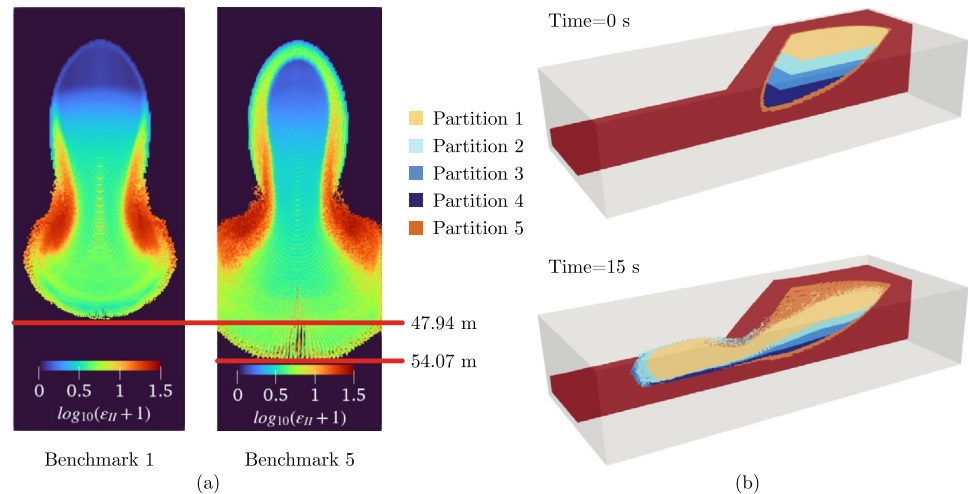


Fig. 29 a Comparison of the final plastic strain distribution and deposition morphology between Benchmark 1 and Benchmark 5. **b** Layered material distribution at the initial and final stages of the simulation along the profile at $y = 13$ m



As shown in Fig. 27a, Benchmark 1 consists of four equal-thickness partitions (each accounting for 25% of the total thickness). To examine the influence of the partition thickness proportion on the sliding response, Benchmarks 2–4 are configured such that the upper-, middle-, and lower-layer properties dominate the overall response, respectively, thereby dominating the overall mechanical response. Figure 27b presents the plan-view morphology of the sliding mass at the end of the simulations. In Benchmark 2, the run-out front reaches $x = 51.08$ m, consistent with the reduced overall shear strength associated with the upper weak partitions.

In addition, a more refined partitioning scheme is employed to highlight the influence of different zoning strategies on the landslide motion characteristics (Benchmark 5). As illustrated in Fig. 28, a new partition is introduced along the failure surface obtained from the SLBL method, to represent a weak layer near the sliding surface. The remaining four partitions maintain identical thickness ratios. The material properties of Partitions 1–4 are assigned in the same manner as in Fig. 25, while Partition 5 adopts the same properties as Partition 1. It should be noted that the total volume of the sliding part remains unchanged, i.e., 1528.87 m^3 .

Due to the presence of the weak layer, the model exhibits a pronounced overall instability once disturbed. The sliding part undergoes persistent shear displacement along the

weak surface, causing the leading edge to advance progressively, with a final run-out distance of about 54.07 m, as illustrated in Fig. 29. Compared with the model without a weak layer (Benchmark 1), the sliding distance increases, the frontal propagation extends further, the depositional fan covers a larger area, and the overall deformation becomes more pronounced. During the motion, the lower part of the sliding mass is dominated by intense shear failure controlled by the weak layer, whereas in the absence of the weak layer, the deformation is characterized by localized failure and a much shorter run-out. The corresponding simulation video is available in the supplementary material. This comparison clearly indicates that the presence of a weak layer significantly enhances the mobility of the landslide mass, leading to a more pronounced and complex dynamic response in the numerical simulations.

The integration of the particle generator with the SLBL method enables parametric studies by converting landslide-model preprocessing into a fast, configuration-driven, and fully reproducible workflow. By controlling geometry, partitioning, and material assignment, it enables practical batch runs while preserving consistent discretization and boundary settings across cases. Although we focused on a two-stage study, the same pipeline extends naturally to richer designs—multi-factor perturbations, inter-partition interactions, spatially varying properties, and stochastic fields

for uncertainty quantification. Under the proposed particle generation framework, the benchmark serves as a compact template for scalable parametric exploration in broader applications, and thus lays the groundwork for more complex investigations.

4.5 Additional remarks

Through extensive testing, we found that both complex geometries and property assignments can be accurately handled within our framework to generate structured material particles. In the evaluation of geometric and property errors, we report the proportion of particles with incorrect assignments. Notably, these erroneous particles are distributed along the model surface, with their maximum distance from the surface being very small—as previously mentioned, no greater than $\sqrt{2}h/4$ in 2D and $\sqrt{3}h/4$ in 3D. In contrast, voxelization method may yield particles located farther from the surface due to the resolution limit imposed by h , although they still remain near the surface. Such deviations are expected to be acceptable for MPM simulations.

The two scenarios presented in Section 4.3 are typically used in geotechnical engineering, and our framework reliably produces high-fidelity discrete representations in such cases. In more advanced applications, however, it may be necessary to incorporate structural elements or more complex multi-component models. To address this, structural or user-defined components can be modeled separately as STL files, discretized independently, and subsequently integrated with the particle set generated from DEM data. All methods within the framework have undergone performance evaluation, and no significant computational overhead has been observed, demonstrating the framework's scalability and efficiency.

For the selection of the optimal grid spacing h , we distinguish between two cases. (i) For problems that are not strongly sensitive to geometric description, we adopt a two-step strategy. First, an initial value of h is estimated from the proposed error bound in terms of the characteristic feature length of the model (shorter edge). Second, a short parameter sweep is carried out by progressive refinement, and h is accepted once the resulting error lies within an acceptable margin relative to the target tolerance. (ii) For problems that are sensitive to geometry and require high-resolution treatment, the optimal choice of h must be determined through a convergence study of the MPM simulations, similar to the work of Peng et al. [23], with the filling mode disabled. In such high-resolution simulations, especially under large deformation, strain localization may cause numerical instabilities. Therefore, the determination of h should be based not only on convergence analysis but also on the physical admissibility of the computed response. In explicit MPM

formulations, the mesh size h is tied to the admissible time step through the Courant–Friedrichs–Lewy (CFL) condition. Moreover, non-local methods should be incorporated into subsequent MPM simulations to eliminate mesh dependency and thereby alleviate the sensitivity to the grid spacing h . However, this lies beyond the scope of the present work.

Our current implementation does not incorporate parallel algorithms or hardware acceleration such as GPUs. Nevertheless, extensive tests demonstrate that even cases involving up to 10^8 particles can be completed efficiently on a laptop. Parallel algorithms can, however, be readily adapted. In the optimized projection-based algorithm, steps 3 and 4 (see Algorithm 1) are inherently parallelizable, as each ray can be processed independently. Furthermore, GPU acceleration should be considered for optimizing the property assignment stage or for adapting new algorithms within the framework, although this requires further investigation in future work.

It is also meaningful to compare our proposed method with existing tools such as *PyCBG* and *CAD2Point*. However, during evaluation, we observed that the current version of *PyCBG* requires users to provide an anonymous function to determine whether a structured material particle within an Axis-Aligned Bounding Box (AABB) resides inside the model, which makes it unsuitable for complex geometries. On the other hand, the C++ implementation of *CAD2Point* is limited to generating unstructured particle discretizations. Therefore, our comparison focuses on the discretization efficiency of three-dimensional models, and interested readers are referred to the *Supplementary Material 3* for further details.

5 Conclusions

This paper has presented an efficient particle generation framework implemented in Julia, designed to produce structured material particles for MPM simulations across multi-context modeling. Supporting 2D and 3D models as well as complex terrains, the framework allows flexible algorithm selection or combination based on scene characteristics and accounts for spatial property partitioning to enable applications involving complex and real-world cases. It has been evaluated on a 2D landslide profile, two generic 3D models, and a complex terrain model. In the 2D case, property boundaries were clearly resolved; in the 3D cases, high-resolution discretization preserved surface textures; and for the terrain model, particles were generated using either geological stratification or SLBL-predicted failure surfaces. Error analysis revealed that geometric accuracy is strongly dependent on the grid spacing h , with property errors becoming more

pronounced in thin-layer regions. Additionally, integrating the particle generator with the SLBL method enables a parametric landslide study that demonstrates our framework's efficient, fully reproducible setup and facilitates investigations into geometric and material uncertainties, multi-factor perturbations, and related extensions.

We established rigorous theoretical estimates for the maximum geometric error in both two-dimensional and three-dimensional configurations, as well as upper bounds for the property error within a three-partitioning scheme. The proposed filling mode achieves particle densification with negligible computational cost. While it may induce a minor artificial volumetric expansion at the model surface, the associated discrepancy is minimal. Even under worst-case scenarios, the maximum offset between misclassified particles and the true model boundary remains within $\sqrt{2}h/4$ in two dimensions or $\sqrt{3}h/4$ in three dimensions. This deviation is particularly insignificant for solid models without internal cavities and can be considered negligible in the context of MPM simulations. Performance evaluations

confirmed that the framework can process millions of particles within 10 s—significantly faster than model file I/O—and that it exhibited no evident computational bottlenecks. The framework avoids volumetric meshing, requiring only watertight surface meshes or equivalent DEM data, supports uniform particle placement per cell, and offers excellent generality and extensibility for preprocessing in MPM and other particle-based methods.

The sloping local base level (SLBL) method

Base level is a concept in geomorphology, representing the lowest surface to which erosion can proceed during landscape evolution. The SLBL builds upon this concept by incorporating the effects of slope undercutting, forming a localized terrain analysis method [16–18]. It can be regarded as an extended form of the base level concept, with a construction principle similar to that of the isobase. Its implementation is straightforward and automated, as shown in Algorithm 2.

Input: Digital Elevation Model (DEM) grid $z_{ij}^{(0)}$;
 Neighbourhood set $\mathcal{N}$ (i.e. 4-neighbour in a raster grid);
 Positive tolerance constant C ;
 Optional slope-angle lower bound $\theta_{\min}$;
 Optional minimum altitude $z_{\min}$;
 Convergence threshold ε or ε_V (volume).
Output: Failure surface $z_{ij}^{(*)}$.

```

 $t \leftarrow 1$ ;
repeat                                     // iterate until convergence
  foreach  $(i, j)$  do                         // loop over every grid node
     $\bar{z} \leftarrow f(z_n^{(t-1)}) \quad n \in \mathcal{N}$ ;
     $z_{ij}^{\text{temp}} \leftarrow \bar{z} - C$ ;
    if  $z_{ij}^{\text{temp}} < z_{ij}^{(t-1)}$  then           // candidate is lower than current surface
      if  $\nabla z_{ij}^{\text{temp}} \geq \theta_{\min} \wedge z_{ij}^{\text{temp}} \geq z_{\min}$  then // optional geometrical/physical checks
         $z_{ij}^{(t)} \leftarrow z_{ij}^{\text{temp}}$ ;
      else
         $z_{ij}^{(t)} \leftarrow z_{ij}^{(t-1)}$ ;
      end
    else
       $z_{ij}^{(t)} \leftarrow z_{ij}^{(t-1)}$ ;
    end
  end
   $t \leftarrow t + 1$ ;
until  $\Delta = \max_{i,j} |z_{ij}^{(t)} - z_{ij}^{(t-1)}| < \varepsilon \vee |\text{Vol}(t) - \text{Vol}(t-1)| < \varepsilon_V$ ;
return  $z_{ij}^{(*)} \leftarrow z_{ij}^{(t-1)}$ ;

```

Algorithm 2 SLBL calculation

The value of C can be estimated from the anticipated geometry of the landslide failure surface along the longitudinal profile:

$$C = 4 \frac{z_{\max}^2}{L_{rh}^2} h^2, \quad (\text{A1})$$

where $z_{\max}$ is the maximum vertical thickness of the profile, L_{rh} is the estimated horizontal extent, and h denotes the grid spacing of the DEM.

Supplementary Information The online version contains supplementary material available at <https://doi.org/10.1007/s00366-025-02222-z>.

Acknowledgements ZH acknowledges the financial support provided by the China Scholarship Council. XZ would like to acknowledge the financial support provided by National Natural Science Foundation of China (Grant No. 52308425).

Author Contributions ZH and XZ wrote the original draft of the paper. ZH, XZ, and XT wrote and revised the final version of the paper. MJ, YP, XZ, and GM supervised this study and provided guidance. ZH developed the first version of the package (MaterialPointGenerator.jl). All authors reviewed and approved the final version of the paper.

Data Availability The framework proposed in this paper is released as an open-source Julia package, *MaterialPointGenerator.jl*, under the MIT license. Further details on installation and usage can be found at <https://github.com/LandslideSIM/MaterialPointGenerator.jl>.

Declarations

Conflict of interest The authors declare no conflict of interest.

References

- Arun PV (2013) A comparative analysis of different dem interpolation methods. *Egypt J Remote Sens Space Sci* 16(2):133–139. <https://doi.org/10.1016/j.ejrs.2013.09.001>
- Bezanson J, Edelman A, Karpinski S, Shah VB (2017) Julia: a fresh approach to numerical computing. *SIAM Rev* 59(1):65–98. <https://doi.org/10.1137/141000671>
- Bertolazzi E, Frego M, Biral F (2020) Point data reconstruction and smoothing using cubic splines and clusterization. *Math Comput Simul* 176:36–56. <https://doi.org/10.1016/j.matcom.2020.04.002>. *Applied Scientific Computing XV: Innovative Modelling and Simulation in Sciences*
- Bezanson J, Karpinski S, Shah VB et al (2018) Microbenchmarks. <https://julialang.org/benchmarks/>
- Cicoira A, Blatny L, Li X, Trottet B, Gaume J (2022) Towards a predictive multi-phase model for alpine mass movements and process cascades. *Eng Geol* 310:106866. <https://doi.org/10.1016/j.enggeo.2022.106866>
- Chandru V, Manohar S, Prakash CE (1995) Voxel-based modeling for layered manufacturing. *IEEE Comput Graph Appl* 15(6):42–47. <https://doi.org/10.1109/38.469516>
- Ceccato F, Yerro A, Di Carluccio G (2024) Simulating landslides with the material point method: best practices, potentialities, and challenges. *Eng Geol* 338:107614. <https://doi.org/10.1016/j.enggeo.2024.107614>
- Dawson-Haggerty M et al (2019) trimesh. <https://trimsh.org/>
- Duncan S, Park N, Ott C, Langlotz T, Regenbrecht H (2021) Voxel-based immersive mixed reality: a framework for ad-hoc immersive storytelling. *Presence* 30:5–29. https://doi.org/10.1162/pres_a_00364
- Eisemann E, Décoret X (2006) Fast scene voxelization and applications. In: *Proceedings of the 2006 symposium on interactive 3D graphics and games, I3D '06*. Association for Computing Machinery, New York, pp 71–78. <https://doi.org/10.1145/1111424>
- Fernández F, Amaral Vargas E, Quadros Velloso R (2020) A 3D discretization procedure for the material point method (MPM). *Comput Part Mech* 7(4):725–733. <https://doi.org/10.1007/s40571-019-00303-7>
- Geuzaine C, Remacle J-F (2009) Gmsh: a 3-D finite element mesh generator with built-in pre- and post-processing facilities. *Int J Numer Methods Eng* 79(11):1309–1331. <https://doi.org/10.1002/nme.2579>
- Habib M (2021) Evaluation of dem interpolation techniques for characterizing terrain roughness. *CATENA* 198:105072. <https://doi.org/10.1016/j.catena.2020.105072>
- Huo Z, Alkhimenkov Y, Jaboyedoff M, Podladchikov Y, Räss L, Wyser E, Mei G (2025) A high-performance backend-agnostic material point method solver in Julia. *Comput Geotech* 183:107189. <https://doi.org/10.1016/j.compgeo.2025.107189>
- Hornung A, Kobbelt L (2006) Robust reconstruction of watertight 3D models from non-uniformly sampled point clouds without normal information. In: *Proceedings of the fourth eurographics symposium on geometry processing*. SGP '06. Eurographics Association, Goslar, pp 41–50
- Jaboyedoff M, Baillifard F, Couture R, Locat J, Locat P (2004) Toward preliminary hazard assessment using DEM topographic analysis and simple mechanical modeling by means of sloping local base level. *Landslides evaluation and stabilization*. Balkema, pp 199–205
- Jaboyedoff M, Chigira M, Arai N, Derron M-H, Rudaz B, Tsou C-Y (2019) Testing a failure surface prediction and deposit reconstruction method for a landslide cluster that occurred during Typhoon Talas (Japan). *Earth Surf Dyn* 7(2):439–458. <https://doi.org/10.5194/esurf-7-439-2019>
- Jaboyedoff M, Carrea D, Derron M-H, Oppikofer T, Penna IM, Rudaz B (2020) A review of methods used to estimate initial landslide failure surface depths and volumes. *Eng Geol* 267:105478. <https://doi.org/10.1016/j.enggeo.2020.105478>
- Jiang C, Schroeder C, Teran J, Stomakhin A, Selle A (2016) The material point method for simulating continuum materials. *Assoc Comput Mach*. <https://doi.org/10.1145/2897826.2927348>
- Liao D (2008) GPU-accelerated multi-valued solid voxelization by slice functions in real time. In: *Proceedings of the 24th spring conference on computer graphics, SCCG '08*. Association for Computing Machinery, New York, pp 113–120. <https://doi.org/10.1145/1921264.1921288>
- Meister D, Ogaki S, Benthin C, Doyle MJ, Guthe M, Bittner J (2021) A survey on bounding volume hierarchies for ray tracing. *Comput Graph Forum* 40(2):683–712. <https://doi.org/10.1111/cgf.142662>
- Perez KA, Barclay PL, Zhang DZ (2024) Nodal force error and its reduction for material point methods. *J Comput Phys* 498:112681. <https://doi.org/10.1016/j.jcp.2023.112681>
- Peng C, Li S, Wu W, An H, Chen X, Ouyang C, Tang H (2022) On three-dimensional SPH modelling of large-scale landslides. *Can Geotech J* 59(1):24–39. <https://doi.org/10.1139/cgj-2020-0774>
- Pan D, Xu Z, Lu X, Zhou L, Li H (2020) 3D scene and geological modeling using integrated multi-source spatial data: methodology, challenges, and suggestions. *Tunn Undergr Space Technol* 100:103393. <https://doi.org/10.1016/j.tust.2020.103393>

25. Rowley C (2022) PythonCall.jl: Python and Julia in harmony. <https://github.com/JuliaPy/PythonCall.jl>
26. Sulsky D, Chen Z, Schreyer HL (1994) A particle method for history-dependent materials. *Comput Methods Appl Mech Eng* 118(1):179–196. [https://doi.org/10.1016/0045-7825\(94\)90112-0](https://doi.org/10.1016/0045-7825(94)90112-0)
27. Shimrat M (1962) Algorithm 112: position of point relative to polygon. *Commun ACM* 5(8):434. <https://doi.org/10.1145/368637.368653>
28. Si H (2015) Tetgen, a Delaunay-based quality tetrahedral mesh generator. *ACM Trans Math Softw* 41(2):11. <https://doi.org/10.1145/2629697>
29. Sacha D, Jérôme D (2021) PyCBG, a python module for generating CB-Geo MPM input files. Zenodo. <https://doi.org/10.5281/zenodo.5179973>
30. Shicai Y, Qianjun W, Rong L (2011) An high efficient and speed algorithm of ray casting in volume rendering. In: 2011 International conference on consumer electronics, communications and networks (CECNet), pp 1027–1030. <https://doi.org/10.1109/CECNET.2011.5768256>
31. Schwarz M, Seidel H-P (2010) Fast parallel surface and solid voxelization on GPUs. *ACM Trans Graph* 29(6):1–10. <https://doi.org/10.1145/1882261.1866201>
32. Szucki M, Suchy JS (2012) A voxelization based mesh generation algorithm for numerical models used in foundry engineering. *Metall Found Eng* 38(1):43–43. <https://doi.org/10.7494/mafe.2012.38.1.43>
33. Stomakhin A, Schroeder C, Chai L, Teran J, Selle A (2013) A material point method for snow simulation. *ACM Trans Graph* 32(4):1–10. <https://doi.org/10.1145/2461912.2461948>
34. Shi B, Zhang Y, Zhang W (2019) Run-out of the 2015 Shenzhen landslide using the material point method with the softening model. *Bull Eng Geol Environ* 78(2):1225–1236. <https://doi.org/10.1007/s10064-017-1167-4>
35. Torres R, Martín PJ, Gavilanes A (2009) Ray casting using a roped BVH with CUDA. In: Proceedings of the 25th spring conference on computer graphics. SCCG '09. Association for Computing Machinery, New York, pp 95–102. <https://doi.org/10.1145/1980462.1980483>
36. Whang K-Y, Song J-W, Chang J-W, Kim J-Y, Cho W-S, Park C-M, Song I-Y (1995) Octree-R: an adaptive octree for efficient ray tracing. *IEEE Trans Vis Comput Graph* 1(4):343–349. <https://doi.org/10.1109/2945.485621>
37. Yin Y, Li B, Wang W, Zhan L, Xue Q, Gao Y, Zhang N, Chen H, Liu T, Li A (2016) Mechanism of the December 2015 catastrophic landslide at the Shenzhen landfill and controlling geotechnical risks of urbanization. *Engineering* 2(2):230–249. <https://doi.org/10.1016/J.ENG.2016.02.005>
38. You Y, Yang J, Zheng X, Xie Y, Lyu J, Osman AS (2025) A 3D material point discretization approach for complex terrain and geological body: Numerical implementation and application. *Comput Geotech* 177:106884. <https://doi.org/10.1016/j.compgeo.2024.106884>
39. Zhang X et al (2023) CAD2Point. <https://comdyn.hy.tsinghua.edu.cn/show/software/602-cad2point>
40. Zhang DZ, Perez KA, Barclay PL (2024) Rapid particle generation from an stl file and related issues in the application of material point methods to complex objects. *Comput Part Mech* 11(5):2291–2305. <https://doi.org/10.1007/s40571-024-00813-z>
41. Zheng XC, Pisanò F, Vardon PJ, Hicks MA (2021) An explicit stabilised material point method for coupled hydromechanical problems in two-phase porous media. *Comput Geotech* 135:104112. <https://doi.org/10.1016/j.compgeo.2021.104112>
42. Zheng XC, Pisanò F, Vardon PJ, Hicks MA (2022) Fully implicit, stabilised, three-field material point method for dynamic coupled problems. *Eng Comput* 38(6):5583–5602. <https://doi.org/10.1007/s00366-022-01678-7>
43. Zheng XC, Seaid M, Pisanò F, Hicks MA, Vardon PJ, Huvaj N, Osman AS (2023) A material point/finite volume method for coupled shallow water flows and large dynamic deformations in seabeds. *Comput Geotech* 162:105673. <https://doi.org/10.1016/j.compgeo.2023.105673>
44. Zhang Q, Zhu H (2018) Collaborative 3d geological modeling analysis based on multi-source data standard. *Eng Geol* 246:233–244. <https://doi.org/10.1016/j.enggeo.2018.10.001>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Terms and Conditions

Springer Nature journal content, brought to you courtesy of Springer Nature Customer Service Center GmbH (“Springer Nature”).

Springer Nature supports a reasonable amount of sharing of research papers by authors, subscribers and authorised users (“Users”), for small-scale personal, non-commercial use provided that all copyright, trade and service marks and other proprietary notices are maintained. By accessing, sharing, receiving or otherwise using the Springer Nature journal content you agree to these terms of use (“Terms”). For these purposes, Springer Nature considers academic use (by researchers and students) to be non-commercial.

These Terms are supplementary and will apply in addition to any applicable website terms and conditions, a relevant site licence or a personal subscription. These Terms will prevail over any conflict or ambiguity with regards to the relevant terms, a site licence or a personal subscription (to the extent of the conflict or ambiguity only). For Creative Commons-licensed articles, the terms of the Creative Commons license used will apply.

We collect and use personal data to provide access to the Springer Nature journal content. We may also use these personal data internally within ResearchGate and Springer Nature and as agreed share it, in an anonymised way, for purposes of tracking, analysis and reporting. We will not otherwise disclose your personal data outside the ResearchGate or the Springer Nature group of companies unless we have your permission as detailed in the Privacy Policy.

While Users may use the Springer Nature journal content for small scale, personal non-commercial use, it is important to note that Users may not:

1. use such content for the purpose of providing other users with access on a regular or large scale basis or as a means to circumvent access control;
2. use such content where to do so would be considered a criminal or statutory offence in any jurisdiction, or gives rise to civil liability, or is otherwise unlawful;
3. falsely or misleadingly imply or suggest endorsement, approval, sponsorship, or association unless explicitly agreed to by Springer Nature in writing;
4. use bots or other automated methods to access the content or redirect messages
5. override any security feature or exclusionary protocol; or
6. share the content in order to create substitute for Springer Nature products or services or a systematic database of Springer Nature journal content.

In line with the restriction against commercial use, Springer Nature does not permit the creation of a product or service that creates revenue, royalties, rent or income from our content or its inclusion as part of a paid for service or for other commercial gain. Springer Nature journal content cannot be used for inter-library loans and librarians may not upload Springer Nature journal content on a large scale into their, or any other, institutional repository.

These terms of use are reviewed regularly and may be amended at any time. Springer Nature is not obligated to publish any information or content on this website and may remove it or features or functionality at our sole discretion, at any time with or without notice. Springer Nature may revoke this licence to you at any time and remove access to any copies of the Springer Nature journal content which have been saved.

To the fullest extent permitted by law, Springer Nature makes no warranties, representations or guarantees to Users, either express or implied with respect to the Springer nature journal content and all parties disclaim and waive any implied warranties or warranties imposed by law, including merchantability or fitness for any particular purpose.

Please note that these rights do not automatically extend to content, data or other material published by Springer Nature that may be licensed from third parties.

If you would like to use or distribute our Springer Nature journal content to a wider audience or on a regular basis or in any other manner not expressly permitted by these Terms, please contact Springer Nature at

onlineservice@springernature.com